FALL PREVENTION AND DETECTION

The implementation of cost-effective technology into a care-home setting.

Abstract

We intend to show that the monitoring and potential prevention of falls may occur with cost-effective technology in a care-home setting. This is done by using the BBC micro:bit, where the accelerometer can inform of an individual's current movement, a pulse-oximeter can inform of an individual's heart rate and oxygen levels, a buzzer unit with an astable that can alert the user and the staff, and an input and output terminal, which allows staff to see the current data and also edit the individual's boundaries for heart rate and oxygen levels and whether they have dementia or not

GROUP – FALLARM:

Umar Sultonov (B18); Hiranya Das (E18); Alex Lodge (B18)

usultonov_b18@kegs.org.uk; hdas_e18@kegs.org.uk; alodge_b18@kegs.org.uk

Contents

Plan ... 3
The early development of the project ... 4
Initial ideas ... 6
Microcontroller research ... 8
Research into vital signs ... 9
Reliability of Optical Sensors ... 11
Design Brief ... 11
Specification ... 11
The Case ... 12
Specification ... 12
Problems during the design ... 12
The decision between the 2 cases ... 13
Images of Case ... 14
The Hardware ... 15
Microcontroller ... 15
Astable and buzzer unit circuit ... 15
Pulse oximeter sensor ... 16
The Algorithm ... 17
Problem with Pulse oximeter ... 17
Development ... 18
Block diagram ... 19
Flowchart for V2 of sensor & receiver code ... 20
Flowchart for V3 of sensor code ... 21
Flowchart for V3 of receiver code ... 22
Version 2 output image ... 23
The website ... 23
Layout ... 24
Implementation into a care-home setting ... 24
Evaluation of project ... 25
Bibliography ... 26
Appendix A: Sensor Code V2 ... 28
Appendix B: Receiver Code V2 ... 34
Appendix C: Sensor Code V3 ... 36
Appendix D: Receiver Code V3 ... 42
Appendix E: (In progress) Website Code ... 45
Appendix F: Roles ... 46

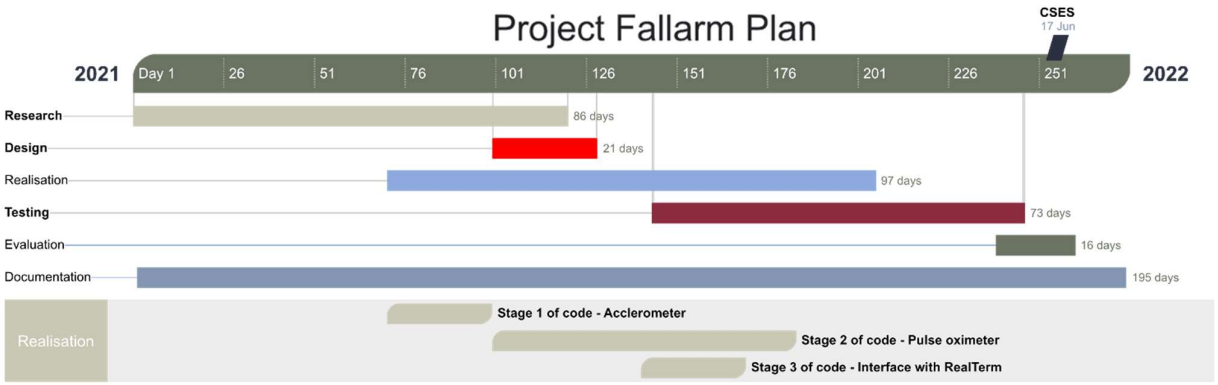
Plan

Projects	Priority	Status	Ease	Epic	Sprint	Engineers
Advanced Website Microbit/PC I/O Terminal	P3	In Progress	Medium	Microbit Interface	Code Design	Hiranya Alex
Pulse Oximeter Sensor	P1	In Progress	Extremely har	Microbit Sensor code	Code Wire	Umar
CODE V3	P3	Complete	Medium	Microbit Sensor code	Code	Umar
Case v1	P1	Complete	Medium	Microbit Assembly	Design	Umar
Documentation	P1	Complete	Medium	Presentation	Document	All
Accelerometer Sensor	P1	Complete	Medium	Microbit Sensor code	Code	Umar
Initial research	P1	Complete	Medium	Initial research	Document	All
Basic Microbit/PC I/O Terminal	P1	Complete	Medium	Microbit Interface	Code	Umar
Alternate Case	P4	Complete	Medium	Microbit Assembly	Design	Hiranya

This was organised into sets of stages:

STAGE 1 - Initial ideas & research	Done	Incl. decision of what to undertake and how to undertake it
STAGE 2 - CODE P1 - Accelerometer	Done	Fine tuning complete. Task
STAGE 3 - CODE P2 - Pulse oximeter	In progress	Code provided for sensor is not compatible with microbit. Task
STAGE 4 - Designing & 3D printing suitable case	Done	2 versions found: Task & alt. Task
STAGE 5 - CODE P3a - Interface using RealTerm	Done	Instructions for setting this up for display is in 'Setup'
STAGE 5 - CODE P3b - Interface using website SERIAL API	In progress	Will need to move both sensor and receiver code to V3

We also included a leniency of 2 weeks for each task in case there were holdups encountered. We chose this plan in order to balance out our time and help to ensure maximum efficiency during our project, so that we completed everything in time for the Chelmsford Science and Engineering Society competition. We specifically dedicated enough time for research as we knew that in order to make a successful product that would effectively prevent and detect falls, we would have to extensively look into vital signs and their effect on the risk of fall. Design was given a short slot due to the simplicity of the specification. Realisation included (among other things) the code and so had to take the longest and therefore testing also had to happen at regular intervals. Documentation was completed throughout the project to ensure a last-minute write-up was mitigated as best as possible). Evaluation was completed at the end of the project and took the least amount of time.



Research	T	86 days	06/10/2021	02/02/2022	100%
Design	T	21 days	13/01/2022	10/02/2022	100%
Realisation	T	97 days	15/12/2021	28/04/2022	85%
Testing	T	73 days	26/02/2022	08/06/2022	85%
Evaluation	T	16 days	01/06/2022	22/06/2022	100%
Documentation	T	195 days	07/10/2021	06/07/2022	100%
CSES	M	0 days	17/06/2022	17/06/2022	100%

The early development of the project

Our project will attempt to prove that low-cost technology can be effectively implemented into adult social care. In order to achieve our aim, we set objectives to achieve, later organised into stages for better organisation. To tackle our aim, we drew up initial ideas which was driven by a group discussion.

The project should be a proof of concept to show that a commercialised version of the product could be used by care homes to allow for care home staff to monitor people more easily, reducing the need for a person supervising a resident of the care home at all times, or improve the quality of life in a care home. The product will need to combat a major problem within the care home to have influence on both staff and residents. By presenting the collected data on the resident to staff, staff can react more effectively to that problem that our product will detect.

We will have achieved this if we:

- Produce a working prototype of a product that can mitigate a particular problem in the care home
    - As this will include communication with staff, our prototype must collect the required data, process it in regard to the problem, and display it in a comprehensive format for the care home staff
    - It must be low-cost
- Described how this product would be best implemented into a care home

Objectives:

- We will need to research into problems affecting care homes
- We will need to decide on what problem to tackle
- We will need to draw up different methods of tackling this problem and evaluate each one, choosing one clear solution
- Research into how to best carry out the project in order to mitigate the problem identified
- Carry out the project continuously referring to the previous research conducted
- Form conclusions, with regard to a real-world implementation and evaluate the project

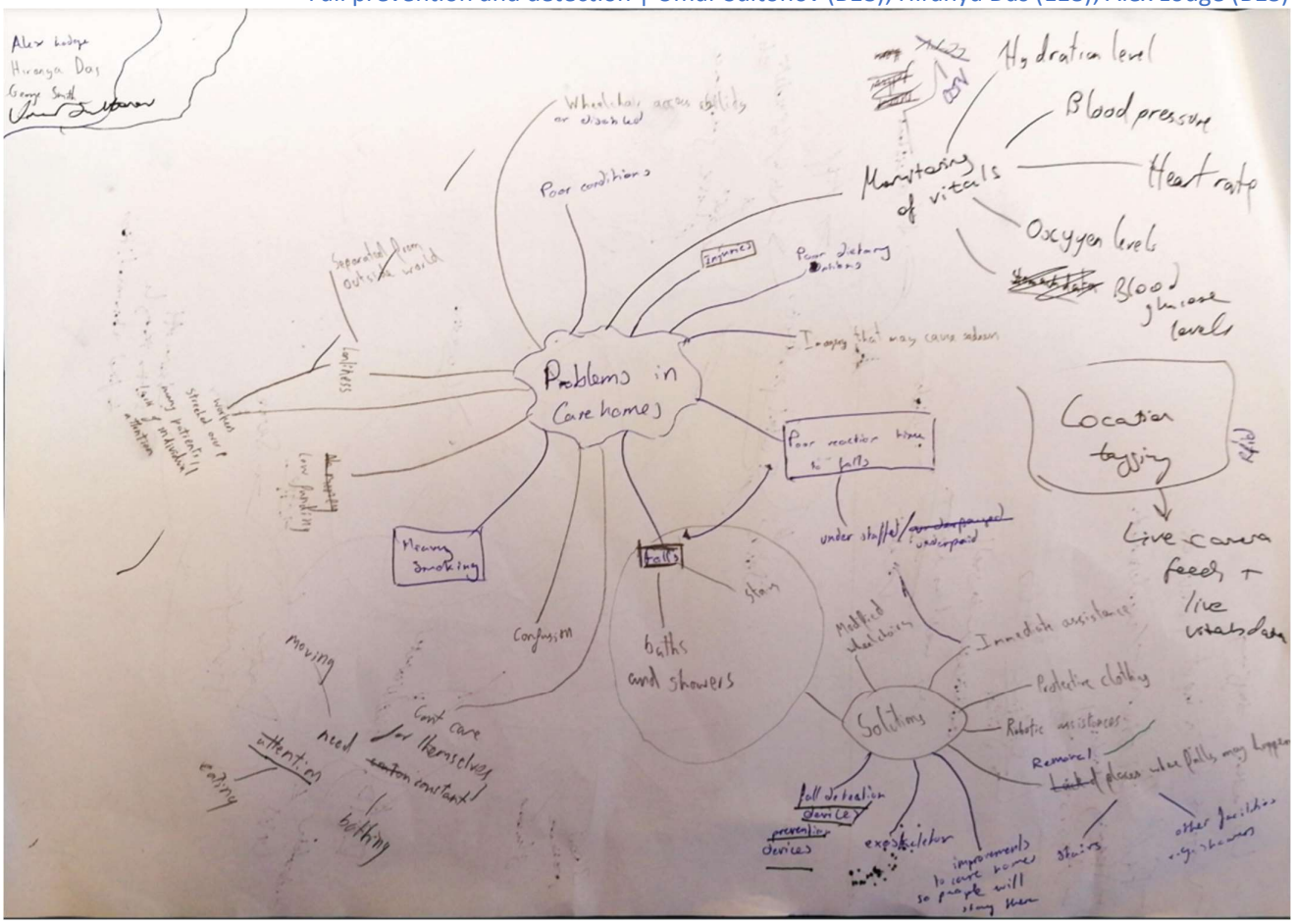


Figure 1 Mind map of problems in care homes

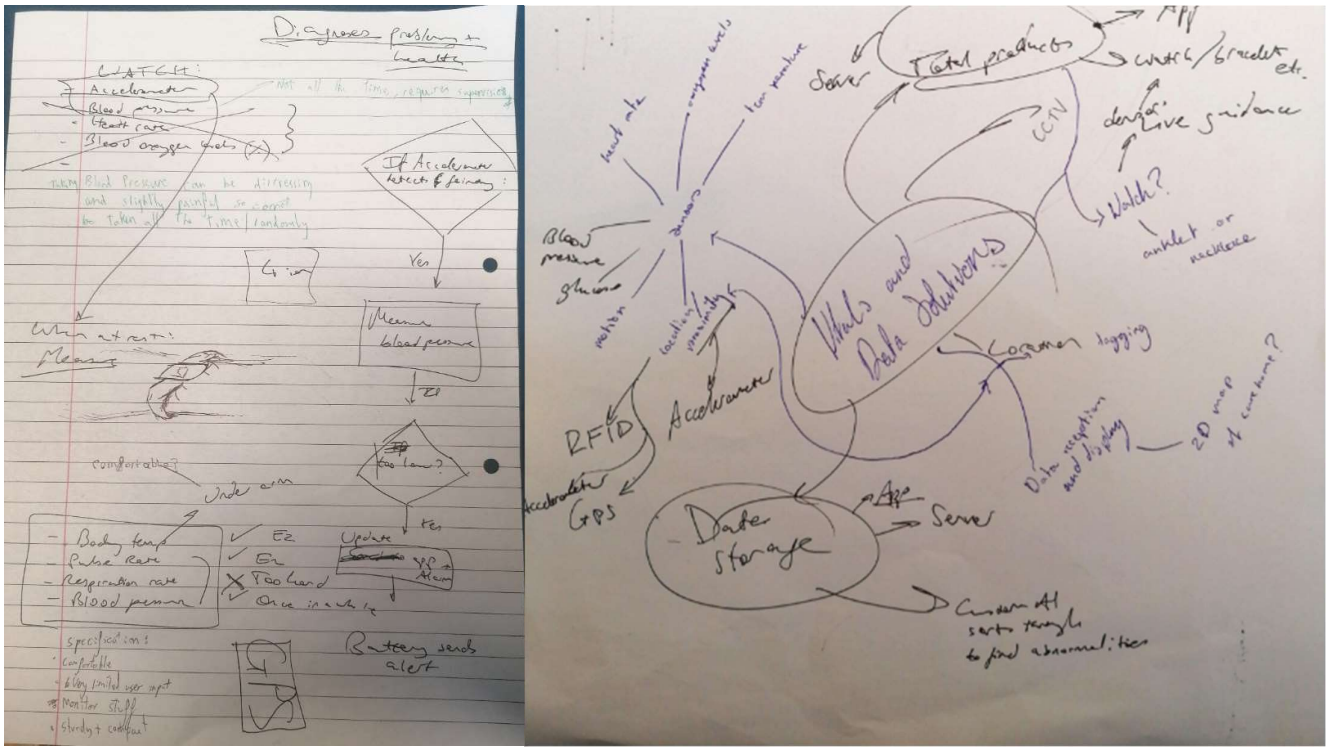


Figure 2a, 2b Initial development of the sensors

Initial ideas

Whilst brainstorming ideas, we had to consider what problems were the most prevalent in a care home, and when deciding which ideas are the best, we referred to how well a particular idea met the problems listed below.

Problem	Solution(s)	Advantages	Disadvantages
Poor reaction time to falls due to understaffing	Have cameras record people and detect if they have fallen. Facial recognition software may also be used in order to pair the accident to an individual	Cameras also accomplish the task of tracking location	- Ethical issues as having the location of a person constantly recorded is an infringement of privacy - Ethical issues as having cameras monitor a person's every move whilst they are in their home and so are entitled to privacy, is not considerate of residents. Facial recognition software also requires data to be stored about the person and needs training with many photos of the person
	Having a way for people to reach out for help if they have fallen	Ensures that there are no false alarms as it is user triggered	Relies on user input, which is not viable considering dementia patients
	Use an accelerometer or gyroscope to detect free fall followed by a period of rest. A way of finding a person quickly and effectively would also reduce reaction time	Works independently of user input so is perfect for dementia patients who may forget they need assistance	May need calibration and tuning to increase sensitivity and specificity
Dementia – will mean confusion and therefore they require assistance	Have an automatic monitor that does not require user input/output to complete its function as a dementia patient would not be able to input the needed data. Having a pressure mat to detect when a person has woken up to inform staff, e.g., to go to the toilet. If you add an accelerometer, it can tackle both these problems		- Residents may purposefully try to avoid a pressure mat so using an accelerometer is more favourable - Increased cost when using a pressure mat to cover the floor around the bed
Location tagging	Having multiple Radio Frequency Identification (RFID) systems installed and using triangulation to find the location of a person. You could also use Global Positioning System (GPS)	- Means that people are monitored so help can arrive quickly in the event of accident	- It would be significantly more expensive as it requires either access to GPS, or many RFID readers, as the strength of the RFID communication will decrease very quickly with distance - Finding the location and presenting it on a map is challenging and may not actually help a care home staff find the person - Ethical issues as having the location of a person constantly recorded is an infringement of privacy
	Have many radio receivers which measure the strength of the signal to determine how far away the radio sending unit is, and	- Means that people are monitored so help can arrive quickly	- It would be significantly more expensive as it requires multiple units installed with radio communication

	then can find location using triangulation	in the event of accident	- Ethical issues as having the location of a person constantly recorded is an infringement of privacy
Wheelchair access	Design a low-cost, mobile ramp for access to stairs	Ensures inclusive design in all previously constructed stairs and alike	This would be a permanent constructed ramp and so must be durable, and in order to keep price down, it should be adjustable to cater to all stairs etc. Having a durable product with multiple joints is challenging, and may not be worth the hassle for care homes
Often need medical help so need a way of detecting when they would need it before it progresses	Monitor as many vitals as possible and send that data to staff	Illnesses and medical conditions can easily be identified before they progress and deteriorate the health too much	Including all vital signs is challenging, as body temperature must be taken orally, rectally, axillary (underarm), by ear, or by forehead. (Johns Hopkins Medicine, 2022). Respiration rate is very hard to measure as it either requires counting the number of times the chest rises or a chest strap, which would be very uncomfortable to wear. Blood pressure is not a viable vital we could measure, as discussed below.

When considering monitoring vitals, we thought of the following ways of monitoring health:

- Heart rate and blood oxygen levels can be measured by an optical pulse oximeter
- Hydration could be measured by measuring sweat with a moisture sensor, or measuring the resistance between 2 leads put on the skin (if a person sweats, resistance will decrease)
    - This would also show body temperature as if someone is hot, they will sweat
- Blood pressure cannot be feasibly measured
    - "Tests have shown that finger and/or wrist blood pressure devices are not as accurate in measuring blood pressure as other types of monitors. In addition, they are more expensive than other monitors." (Johns Hopkins Medicine, 2022)
    - There are many requirements that must be met before you take a blood pressure reading, like not smoking and drinking, going to the bathroom, relaxing, etc. (Johns Hopkins Medicine, 2022)
- Temperature can be measured using a thermistor and preset resistor in a potential divider circuit connected to an Operational Amplifier to act as an Analogue to Digital Converter (ADC)

Following further research into the most detrimental problems in a care home, we found that, according to World Health Organization (2021), "an estimated 684 000 fatal falls occur each year, making it the second leading cause of unintentional injury death, after road traffic injuries". Furthermore, "Every 15 seconds, an older adult is admitted to the emergency room for a fall. A senior dies from falling every 29 minutes." (Smith, 2018).

Following discussion about the common problems in care homes, we decided to focus on detecting and preventing falls. This would decrease the time it takes to find a fallen patient, increasing chance of survival after a fall and may also combat understaffing in care homes, as staff could remotely monitor multiple people at once.

We also decided on a watch design, with several sensors (heart rate, blood oxygen levels and an accelerometer) helping to complete the task of attempting to not only detect falls but also predict when a fall may be about to occur. This would not only combat the problem of poor reaction to falls, but also vitals could be monitored to allow for on-demand readings to check for other health issues to give warnings for deterioration of their health before it has adverse consequences. As an inclusive design, we made sure that our project would not rely too heavily on input or output from the patient as many may have dementia, Alzheimer's etc., and so our project would need an automatic fall sensor. A watch design made sure that the elderly felt comfortable wearing it without it intruding into the person's comfort, privacy, and happiness in general.

As part of further research into falls, we found that "whilst it is acknowledged that the risk factors in falls include loss of balance, being distracted, environmental hazards etc., there are still biological factors related to vital signs, and this is emphasised through the change in blood pressure and heart rate after falling." (Phelan et al., The Medical clinics of North America, 2015). When considering the link between biological factors and falls, it became apparent that to predict falls, vital signs would need to be measured.

This gave our product the ability to complete the problem regarding medical health, the problem about risk of falls and the problem of people with dementia not being able to call for help easily. We could have chosen to only focus on tackling the problem regarding medical health.
Advantages of using this method of tackling medical health:

- It would mean that care home staff can keep track of all people and their medical history
- Care home staff can take action quickly meaning that the health of the person does not deteriorate and cause more problems.
- An automatic diagnosis could be made, and the related symptom(s) of the diagnosis would be shown to staff to check what problem is most likely

Disadvantages of using this method:

- This would have meant for our project that it had many more sensors, and so we may have chosen to use an Arduino or raspberry pi due to the increased number of pins.
- It would be extremely uncomfortable for the resident to wear this many sensors and would hinder their quality of life, which would be unethical

Since we didn't choose this method, we did not necessarily have to choose another board, only for size

Microcontroller research

Board	Dimensions	# of digital I/O pins	# of analogue I/O	Bluetooth	Wi-Fi	Accel. inbuilt	Code used	Working voltage
Pi zero w	30mm x 65mm x 13mm	40	0				C, C++, Python	5V
Pi zero 2 w	30mm x 65mm x 13mm	40	0				C, C++, Python	5V
Pi zero wh	30mm x 65mm x 13mm	40	0				C, C++, Python	5V
Arduino Fio	66.0 mm × 27.9 mm	14 (6 are PWM)	8	Can be easily connected with modules if not inbuilt as there are analogue I/O (input and output) pins and PWM (pulse width modulation – a way of getting analog waveforms using digital ones.			C++	3.3 V
Arduino Nano	43.18 mm × 18.54 mm	14 (6 are PWM)	8				C++	5 V
LilyPad Arduino	51 mm ⌀	14 (6 are PWM)	6				C++	2.7-5.5 V
Arduino Micro	17.8 mm × 48.3 mm	20 (7 are PWM)	12				C++	5 V
Arduino Pro Mini	17.8 mm × 33.0 mm	14 (6 are PWM)	6				C++	3.3 V / 5 V

Data from: (Wikipedia, n.d.) and (Wikipedia, n.d.)

According to our research into smaller microcontroller boards, we determined that the best option for us would be to use the micro:bit as we hadn't chosen the task of using many sensors to track a resident's health and due to the fact that it is simpler to use and also the fact that you can program it in either blocks, JavaScript, or Python (we used JavaScript). Another microcontroller we were considering was the ESP32, which has WiFi, and has an analogue to digital converter so it can make sense of whatever sensors we connect to it. We could also power it with batteries, and it has a low power consumption. It also has an on board temperature sensor and we could add the other

sensors. However, it would also mean that the time taken for the realisation of the product would exceed our deadlines and therefore, we would get less of the specification done. We therefore stuck with micro:bit to ensure that we met deadlines.

Research into vital signs

Chart 1. Comparison of the heart rate variation between Fallers and Controls

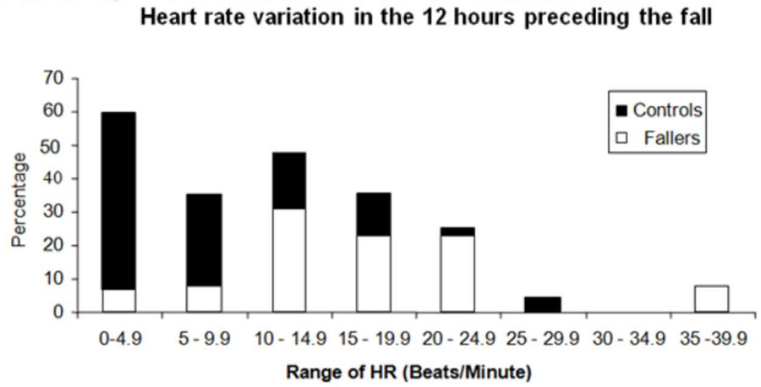
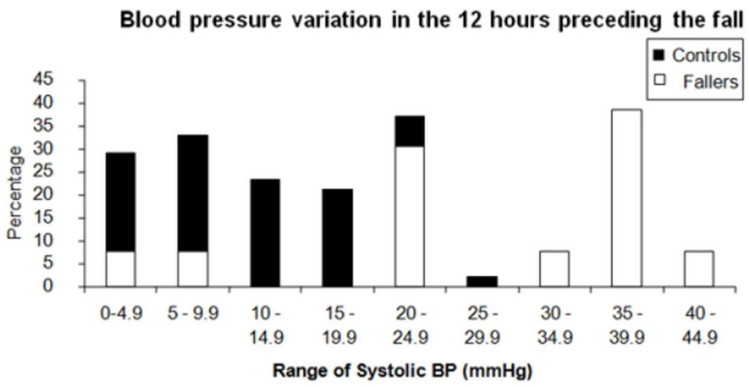


Chart 2. Comparison of the systolic blood pressure variation between Fallers and Controls



1) Blood oxygen levels

According to Cauley et al. (2014), "Hypoxia during sleep may be a risk factor for falls and fractures in older men. Interventions aimed at decreasing nocturnal hypoxia may decrease falls and fractures", which for our project means that, nocturnal hypoxia may be a precursor to a fall, so by detecting it using the pulse oximeter, we can reduce the likelihood of a fall occurring ("men with 10% or more of sleep time with arterial oxygen saturation of less than 90% had a greater risk of having one or more falls (relative risk (RR) = 1.25)"). For our project, this means that the risk of a fall occurring is 25% times more likely if a person has slept for ≥10% of sleep time with oxygen saturation (SpO_2) of <90%.

Oxygen saturation is essentially the percentage of Haemoglobin molecules that are oxygenated.

2) Pulse rate

According to Freilich & Barker (2009), "Cardiovascular variation as measured by increasing range is an acute predictor of falls risk despite relatively normal absolute values", meaning that heart rate of fallers can have a 20% change, compared to a normal 10% variation, helping to predict if a person will/has fallen. The precise numbers of the range of heart rate are 15 ± 10, which means that a variability of 5-25 bpm would suggest that a fall is likely. "We have chosen to primarily measure variability by the range as it is a simple calculation that can be done by anyone on a ward" means that in order for us to use the exact values, it is best, to use a simple range calculation to determine pulse variability.

3) Atrial fibrillation

"Older adults who suffer a fall are twice as likely to have a common type of irregular heartbeat known as atrial fibrillation, according to a new study. Almost 30 percent of those with atrial fibrillation had fallen over the past year compared to about 20 percent of those without atrial fibrillation, the researchers reported in Age and Ageing." (Doyle, 2015) This tells us that the pulse rate of the patient is even more vital to measure, as illnesses such as atrial fibrillation, an irregular heartbeat, can increase the likelihood of falls by a significant amount, with a common symptom of atrial fibrillation being feinting (often a cause of falls), and falling on its own being an even more common symptom when it comes to patients with atrial fibrillation.

4) Blood pressure

Similar to the pulse rate variation, before a fall, you would see around a 20% variation in blood pressure. You can also inform about an individual's hydration levels, as "Dehydration can lead to low blood pressure by reducing your blood volume." (Mitton, 2021). Furthermore, you could measure Postural hypotension with

blood pressure. “Postural hypotension is defined as a reduction in systolic blood pressure of at least 20 mm Hg or in diastolic blood pressure of at least 10 mm Hg within 3 minutes of standing. Postural hypotension affects approximately 30% of community-dwelling older adults and is a fall risk factor. Patients may experience light-headedness, blurred vision, headache, fatigue, weakness, or syncope within 1 to several minutes of standing up, or they may be asymptomatic.” (Phelan et al., The Medical clinics of North America, 2015). Hypotension also “directly increases the risk of heart attack, heart failure, and stroke” (Johns Hopkins Medicine, 2022). This means that a reduction of systolic blood pressure of at least 20mm Hg or diastolic blood pressure of at least 10 mm Hg is a precursor to a fall as well as being a risk factor heart attack, stroke, and heart failure and so should be measured. However, despite the benefits of measuring blood pressure, it would be incredibly inconvenient as it would be very uncomfortable to wear and you cannot get on-demand readings easily, e.g., to measure up until 3 minutes of standing has been reached.

5) Temperature
   As this had no bearing on falls, and a temperature taken at the wrist/fingertip being very inaccurate and not properly representative of body temperature, we have disregarded temperature; “temperature variation (n=13) was minimal with 69% having less than 1 degree Celsius change and 31% having change of 1 to 1.5 degrees Celsius” (Freilich & Barker, 2009)
6) Respiratory rate
   This would be very uncomfortable to measure as it would primarily include a chest-strap and would have no bearing on determining whether a fall will or has taken place: “Recorded respiratory rate variation was also minor (n=13) with 85% having a maximum change of up to 4 breaths per minute.” (Freilich & Barker, 2009)
7) Ambient temperature
   Although body temperature should be disregarded due to inaccuracies, ambient temperature could inform us whether an individual is likely to suffer a stroke as 32°C – 40°C means you may experience heat cramps and exhaustion, 40°C – 54°C means heat exhaustion is more likely, and a temperature of above 54°C often leads to heatstroke. (Healthline, 2017)

8) Dementia
   “There are different personal risk factors that cause people to fall, however, people with dementia are at greater risk because they:
   - are more likely to experience problems with mobility, balance and muscle weakness
   - can have difficulties with their memory and finding their way around
   - can have difficulties processing what they see and reacting to situations
   - may take medicines that make them drowsy, dizzy or lower their blood pressure
   - are at greater risk of feeling depressed
   - may find it difficult to communicate their worries, needs or feelings”

   (NHS inform, 2021)

   From this information we can see that people with dementia are highly likely to have falls, and so quite a few of our patients may have this illness, therefore we compensated for this with our later casing designs, as well as our code, in order to make sure that our product is fiddle free and will be both safe and comfortable for elderly patients, especially those with dementia , which make up around 70% of care home patients (Alzheimer's Society, 2022), showing the necessity to implement some factors to help dementia patients within our design.

We summarised the information regarding vital signs using the following document, considering the 3 vital signs (not including ambient temperature) and the viability of using them in our product.

VITAL SENSOR	IMPORTANCE	RISK FACTORS FOR FALLS	CHANGES AFTER FALL	PREFERRED POSITIONING	WRIST?	MEASURING REQUIREMENTS	AVAILABILITY/PRODUCT	VIABILITY
PULSE RATE	High	None	20% change	Under thumb - radial artery, under neck		None	Pulse Oximeter	
OXIMETER	High	Hypoxia during sleep ∴Measure during sleep	None	fingertip - with cord connecting to watch		Accuracy: Arm + Wrist must be at heart level	Pulse Oximeter	
BLOOD PRESSURE	High	Postural hypotension	20% change	arm - although we can use radial artery		Accuracy: Arm + Wrist must be at heart level		

Reliability of Optical Sensors

As we were going to base our product around the wrist, we felt it was important that research was conducted into Optical heart rate sensors and their viability. As we were using an optical pulse oximeter, it is appropriate to research the accuracy and reliability of these commercial, wrist-worn sensors. One study showed that 'commonly used optical heart rate sensors, such as the ones used herein, generally produce accurate heart rate readings irrespective of the age of the user. However, users should avoid relying entirely on these readings to indicate exercise intensities, as these devices have a tendency to produce erroneous, extreme readings'. (Chow H & Yang C, 2020) This shows that optical heart rate sensors are generally accurate but should not be used in exercise and an awareness of extreme readings is required. Therefore, in our project, we have adapted the algorithm to not include pulse readings or SpO_2 readings in evaluating whether a fall has occurred or not when a person is exercising, which is over 300 seconds (5 minutes) of continuous movement. By continuous, if 4 out of the 5 last values are 'moving', then the algorithm will log continuous movement.

It is also noted that "Fitness wearables offer unreliable readings because they're worn on the wrist, where they're free to move around and won't necessarily be assessing the same blood supply as you'd find on a fingertip." (Munn & Thomas, 2021). Due to this, we have chosen to take the pulse oximeter readings from the finger rather than the wrist, ensuring the most accurate results where it matters the most.

According to Munn & Thomas (2021), "because they're [wrist pulse oximeters] susceptible to giving unreliable results, it's best to look out for trends in your blood sats rather than fixating on individual readings. In other words, if you notice unusually low readings compared to your baseline, then it's worth taking note." Due to this, we have also included an input terminal where care home staff (if needed – a default value is pre-set) can set the upper and lower bounds at which the algorithm would count the oxygen saturation level as abnormal.

The pulse oximeter would be overly sensitive to movement of the arm, therefore, using the accelerometer, we will detect if it is acceptable to take a reading and get a clear result.

The pulse oximeter works by having a light source on one side of the finger and light detectors on the other side. Oxygenated haemoglobin and deoxygenated haemoglobin absorb different amounts of light of different wavelengths. The 2 wavelengths that the pulse oximeter emits is 650nm (red light) and 950nm (infrared light). The pulse oximeter then works out the oxygen saturation by comparing how much infrared light has been absorbed by the blood to the amount of red light absorbed. The precision of these results in increased by having it calibrated to take into account ambient light levels, finger size, and the rest of the tissue absorbing light. (How Equipment Works, n.d.) To find the heart rate, most wearables use photoplethysmography (PPG), which is when you shine light into the skin and measure the amount scattered by blood flow. (Kraudel, 2021) As fast-paced movements disrupt readings as the pulse oximeter could be pushed around (Palladino, 2017), readings will not be processed (still displayed) if the accelerometer returns a continually moving state.

Design Brief

Our product would be a wearable monitor for falls, whilst also monitoring pulse rate and blood oxygen levels. The wearable monitor should then output the relevant data to staff, having already processed what is and is not abnormal for that person.

Specification

- The product must monitor movement
    - It must do this by checking if a person has an acceleration between -600 and 600mg and then is stationary
    - If the person moves continuously (if 4 out of the 5 last values are 'moving'), it should be reset
- The product must monitor pulse rate and oxygen levels in blood
- The product must output to staff in a comprehensive format

- The product must be encased and should be wearable
- Ergonomics:
    - Design must be user-friendly, i.e., case should be smooth with rounded corners and buttons should be easily accessible
    - Case should not be too large and should fit on the wrist, and should be easily detachable by staff, but not accidentally by individuals with dementia
    - Buttons should be toggleable for patients with dementia, or similar illnesses, to prevent accidental triggering of the system
    - Buttons should be clearly labelled so users know the functions of each button

The Case

There were 2 versions of the case which we designed. Both were guided by the same set of specification points. We decided on one and 3D-printed it using the school's 3D printers.

Specification

- Conform to required dimensions, therefore comfortably fits the following:
    - Micro:bit - no rattling to ensure accurate accelerometer readings
    - Battery
    - Lid (either bolted on or just sits on the box)
- Allows for a wire to go to the pulse oximeter
- Have cut-outs for the built-in buttons on the micro:bit
- Have a buzzer to inform user that fall has been detected (Button B used to dismiss this)
- Be fiddle free
    - A feedback point for our case was to make it fiddle free for dementia patients
- Have a connector for the pulse oximeter
    - Due to its uncomfortable application on the thumb, it is best to have it detachable when not in use, i.e., when a staff member is monitoring the person

The alternate case design had the following:

- This one will have one strap which should accommodate: - a compartment for the microbit + header and associated wiring (header pins will need to be broken off) - battery pack (horizontally fixed on the strap - perpendicular)
- The design has the benefit of it being easier to change batteries as they are outside of the case
- The specification remains the same but with the added point of it being ergonomic (fiddle-free for people with dementia), thinner and having the pulse oximeter easy to attach/detach by staff.

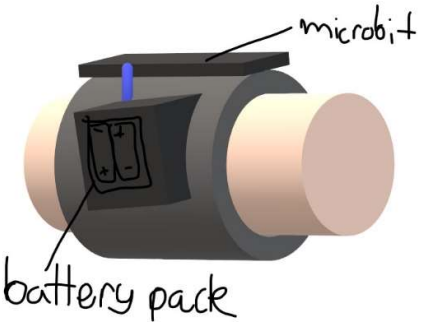


Figure 3 This is a quick sketch of what it would have looked like if we did go ahead with the alternate case

- However, we chose to go ahead with another design which was designed in TinkerCAD, and 3D printed in PLA using the school's 3D printers.

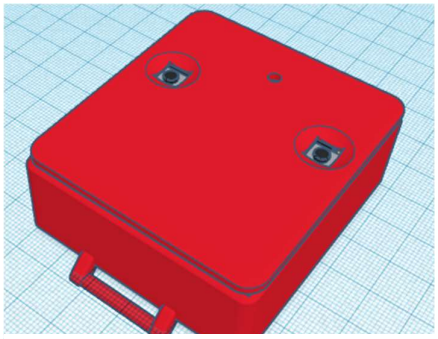


Figure 4 Our current design, featuring all the specification points and housing our current project

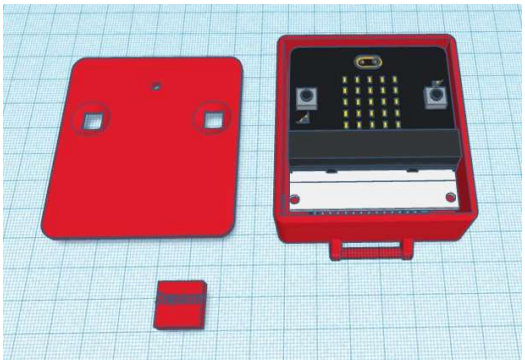
Problems during the design

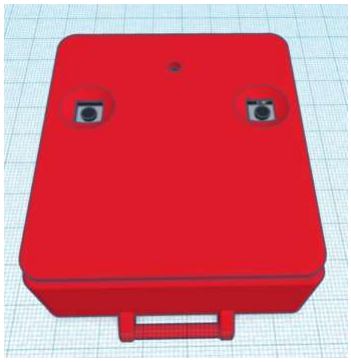
Initially, we began designing using SketchUp, a popular CAD. However, SketchUp was quite limiting, in the sense that it was much harder to alter dimensions and was much harder to incorporate detail into. From

SketchUp, we decided to move to TinkerCAD, a much more user-friendly software that allowed us to easily change dimensions and incorporate finer details, such as the indents for the buttons. This detail was particularly important as indents allowed our users to easily be able to press the buttons, which are quite small. We were also able to include strap holders on the sides of the watch, allowing it to be attached comfortably to the wrist, arm, or another preferred location. By adding strap holders to the watch, we make sure that dementia patients will not accidentally take the case off, but staff could take it off when needed.

The decision between the 2 cases

Our two main designs were both focused on fitting all of the planned components into a comfortable, compact design that was not easy to break/separate. Our first design (Figure 3) was larger than our second design (Figures 4-6), but much more compact and easier to make. As well as this, its key benefit was that it was one single compartment, whereas our second design had multiple compartments that could be broken. Whilst the second design was thinner and smaller than the first and provided a separate storage space to access the batteries, it only saved around 5mm of height, which did not have a significant impact on the overall product. A bulkier case would always mean that it would be uncomfortable to wear, causing annoyance among the residents of the care homes as well as dementia patients fidgeting with the case. Therefore, we decided to develop the first design for our final product. If we had chosen Figure 3, it would effectively be another box on their arm as a battery pack, so this would greatly reduce the design's suitability for care home use.





Houses micro:bit, header board, battery pack, and buzzer

Figure 5a and 5b Current case

Our mentor Dr Erika Sanchez-Velazquez advised us of the potential pitfalls of first design and the feedback given was implemented into our set of specification points:

- Dementia patients will fiddle with the case - not viable to keep there for extended periods of time
  - Due to this feedback point, we ensured that there was a detachable strap. We decided that Velcro would work for this as it is simple, but means it is not accidentally taken off
- A connector for the pulse oximeter is useful to be used as having a pulse oximeter on their thumb constantly may annoy patients
  - Due to this, we ensured to include a connector for the pulse oximeter using the female-female connector

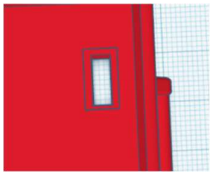


Figure 6 The connector for the pulse oximeter would consist of 4 female to female connectors which would be permanently attached to the pulse oximeter but attached at the case

Images of Case

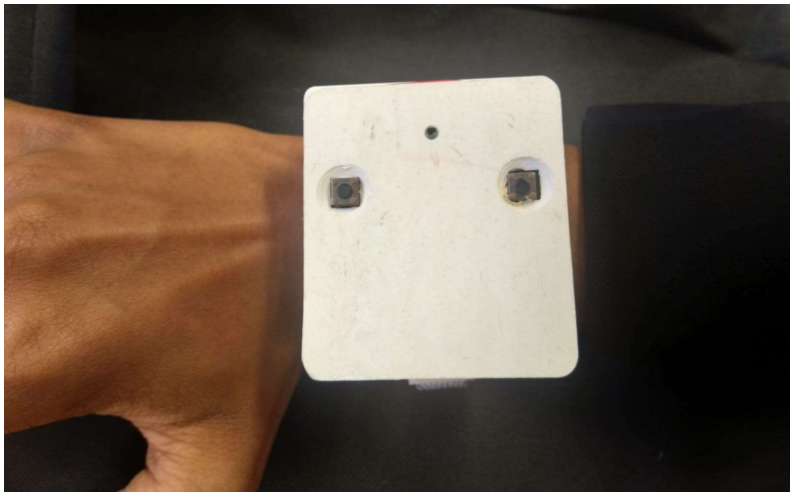


Figure 7 The case on an arm with Velcro attached

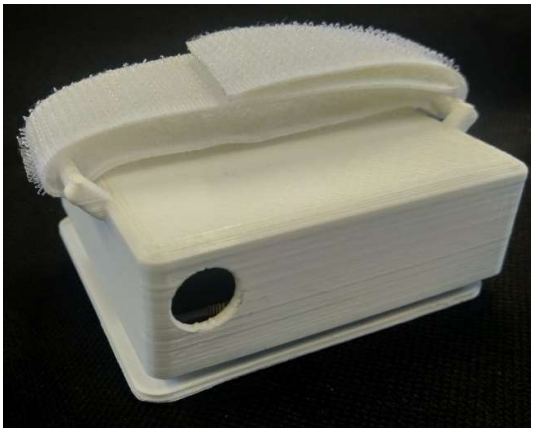


Figure 8 Bottom of case with Velcro strap

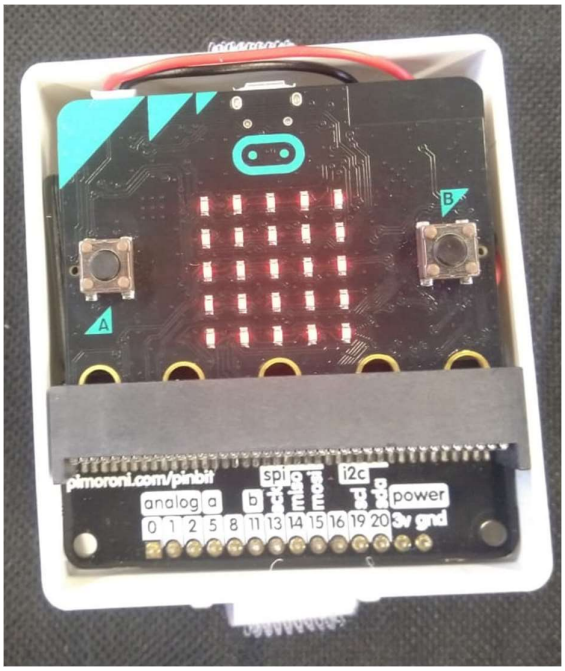


Figure 9 The case with the lid lifted off. Here, as the LEDs are on, fall has been triggered

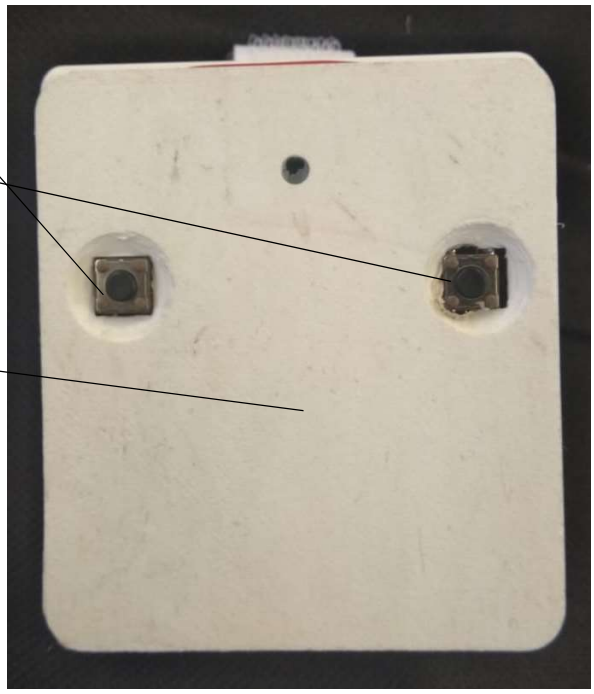


Figure 10 Top of case with cut-outs and buttons visible.

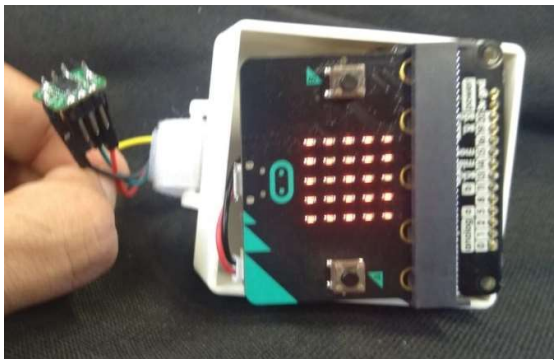


Figure 11 The pulse oximeter and micro:bit connected with female-to-female jumper wires (micro:bit isn't properly inserted into the case)

The pins that fit through the cut-out to connect to the pulse oximeter

Figure 12 (Left) Micro:bit with its header board and the pulse oximeter with the header pins soldered on

The Hardware

Microcontroller

We chose to use the micro:bit for our microcontroller, in conjunction with the MAXREFDES117# pulse oximeter. This meant that there were compatibility issues with the code the manufacturer provided, and the interrupt pin was not an obvious connection to be made to the micro:bit. Therefore, we could not get the pulse oximeter to take the appropriate readings and interface with the micro:bit.

The micro:bit was favoured as it already had many of the specification points we needed:

- Radio or Wi-Fi capabilities (large radius needed)
- Analogue to Digital Converter (ADC)
- Enough pins for:
  - Pulse oximeter
  - 2 buttons
  - Output for a buzzer
- Compact
- Can be powered by a battery
- As simple (language and hardware) to use as possible
- Sensors for the microcontroller are readily available

Other microcontrollers were considered but these had a steep learning curve. The ESP32, for example, was considered for its Wi-Fi connectivity, its Analogue to Digital Converter (therefore can use analogue sensors easily), its small size, and its low power consumption. However, due to the requirement of C++, we decided against this alternative.

Astable and buzzer unit circuit

Whilst we can use the micro:bit sensor algorithm to output a pulsing waveform with a specific period, we have assembled an electronic replication of the alarm that the watch would use to alert staff. This is a way of showing what really happens under all of the abstraction that the micro:bit does for us. This uses a 555 astable timer circuit, which provides the pulses, a MOSFET, to act as a transducer driver for the buzzer, and the buzzer itself.

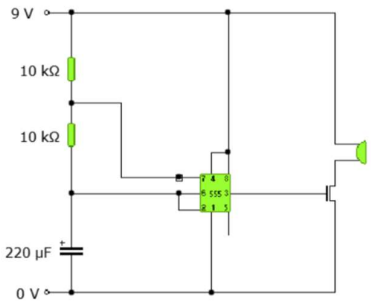


Figure 13 Astable circuit with n-channel MOSFET

Initially, I decided to use a Monostable circuit for my design, due to it allowing a button to be pressed to activate the buzzer, which is beneficial as it allows the elderly person to signify if they have had a fall, therefore allowing for the care home workers to know when a fall has occurred, even if the other sensors have not picked it up (i.e they are stuck in a chair and unable to get up). However, the disadvantages of this is that for a monostable circuit, the buzzer does not continually sound, and so will only sound once when the button is initially pressed, and for a short period of time after it is pressed. This would cause less awareness of when the patients have fallen over, as the care home staff may not hear the short buzzer noise. As well as this, it was required that the buzzer circuit is sounded when pin 0 of the micro:bit went high, which is not something that a monostable can do. Therefore, I chose an Astable circuit, which has several advantages over the monostable, one of the most vital being that it repeatedly buzzes in order to alert the care home workers when a person has fallen over, and to ensure that it is heard, as the noise may be covered up occasionally. Another advantage is that the astable requires less human interaction and will sound whenever a fall is detected, and so for people with dementia (which is a large majority of patients in care homes), the button may be tempting to fiddle with which could cause a false alarm, which would mean that the button press could prevent staff from being where they need to be due to them attending to a false alarm.

In the circuit:

- A buzzer should alternate between sounding and not sounding to act as an alarm
- The resistors should be at least 1kΩ each
- The wires on the breadboard should be colour coded (+ve, -ve, etc.)
- The 555 clock should produce a digital square wave
- The buzzer should sound for 3 seconds (± 0.5 seconds)
- The FET should saturate at the positive power rail, reducing the voltage lost to heating up the transistor, and potentially damaging it

There were previous discussions of using an LDR with a potential divider to detect when a person should be sleeping, but is moving according to the accelerometer, but this is ineffective and can be easily determined by staff as the accelerometer state will be printed to them anyway.

During the building of this circuit, I encountered several problems that needed to be solved, with an example being that I was unable to achieve the correct frequency for my astable build, and therefore, was unable to successfully fulfil the timings required in my specification. In order to overcome this issue, I looked at the equation for the mark time of the astable (the time in which the buzzer would be sounding) and found that the mark time could be increased by either increasing the resistance in the circuit, or by increasing the capacitance of the circuit. The equation, $0.7 \times (\mathrm{R1} + \mathrm{R2}) \times \mathrm{C}$, shows how both the resistance and capacitance affect the mark time of the circuit by increasing it through multiplication. At first, I decided to change the resistance, as this seemed to be the easiest and most simple option to achieve my goal, however I encountered an issue when doing this, as the resistors seemed to have either too big or too little an impact on the mark time, and I could therefore not get it to fit the specification I had created. This proved to be a simple issue however to fix, as I decided to instead change the capacitance within the circuit, by increasing the capacitor value from 22 μF to 220 μF, which theoretically would multiply the mark time by 10, which would enable the buzzer to go from sounding for 0.3 seconds, to sounding for 3 seconds. When implemented, this was successful at doing what I had expected, and so I was able to meet the specification for this aspect of the circuit.

In the future, this design could be improved through the use of more accurate timing, and through the use of a more compact design, due to it taking up quite a significant area when prototyping, causing a less ergonomic case in the end. Therefore, the design could be improved by making a more compact, and easier to interact with design, in order to make sure that it is comfortable for the elderly patient to wear. I have learnt how to use a FET to ensure that the buzzer sounds and is not quietened by not having enough voltage going into it from 555 clock circuit, which guarantees that it will be heard by both the elderly patient and care home workers around them. Another thing that I have learnt from this project, is how to connect and wire a 555 clock configured as an astable, and how to change the resistor and capacitor values to ensure that the frequency of the astable is appropriate.

Pulse oximeter sensor

https://uk.rs-online.com/web/p/sensor-development-tools/1914234?cjevent=204a634d228b4c3dd24130004903c32797c00cdf9f2cc20e7&cmmmc=UK-CJAFF--JVWEB+PERFORMANCE-_-UK+Homepage

https://shop.pimoroni.com/products/max30101-breakout-heart-rate-oximeter-smoke-sensor?variant=21482065985619

https://coolcomponents.co.uk/products/pulse-oximeter-and-heart-rate-sensor-max30101-max32664-qwiic?currency=GBP&variant=30181692932157&utm_medium=cpc&utm_source=google&utm_campaign=Google%20Shopping&gclid=Cj0KCQiAmpyRBhC-ARIsABs2EAoqLqk9zlKTTpRGj0kvqT_yUlUmXyOhZk-CE1dvD359hjKtNrFjSKIaArS0EALw_wcB

https://www.ebay.co.uk/itm/142639474347?chn=ps&norover=1&mkevt=1&mkrid=710-134428-41853-0&mkcid=2&itemid=142639474347&targetid=1278608952136&device=m&mktype=&googleloc=1006748&poi=&campaignid=14727339348&mkgroupid=127909237815&rlsatarget=pla-

1278608952136&abcId=9300672&merchantid=109732774&gclid=CjwKCAiAvaGRBhBlEiwAiY-yMAtA7l_HW0EqzV6H15vAQpWzSfGvu7g-pZMh7GsCdDu5jNnCrwBL6BoCf1kQAvD_BwE

From the above sensors, we chose the first (rs-online) as it could be powered by the micro:bit's 3V output and ran on mbed, the same platform micro:bit runs on. It also followed the specification we outlined for which sensors we could use:

- Small size
- micro:bit-friendly
- Is related to falls (causes/is a precursor to falls is better)
- Is easy to get readings on demand (does not bother the wearer)
- Does not need much supervision
- Does not use much power
- The area which you need to measure to get the readings must be near to the area most of the other sensors use/wrist

Image of pulse oximeter we decided to use:

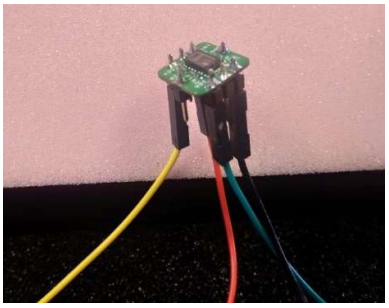


Figure 14 Pulse oximeter with female-to-female jumper leads connected

LEDs to find oxygen saturation

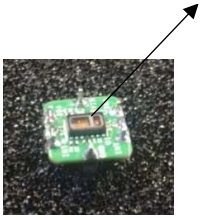


Figure 15 Pulse oximeter

The Algorithm

Problem with Pulse oximeter

The pulse oximeter has example source code available for both Arduino and mbed platforms (micro:bit is an mbed platform). The MAXREFDES117 (the pulse oximeter) has 3 C++ files and 3 header files: algorithm.cpp, MAX30102.cpp, and main.cpp. This code from the manufacturer is specifically designed for mbed platforms: • Maxim Integrated MAX32600MBED# • Freescale FRDM-K64F • Freescale FRDM-KL25Z. However, as micro:bit is an mbed platform as well (same version number), and the sensor should be able to be powered by the micro:bit's 3V pin voltage, I tried using the micro:bit, but the mbed compiler shows the following errors:

- As provided (unchanged code from above link), the compiler states: "Error: Identifier "I2C_SDA" is undefined in "MAX30102/MAX30102.cpp", Line: 65, Col: 10". According to [link](https://maximsupport.microsoftcrmportals.com/en-us/knowledgebase/article/000100869), using a different board to what is suggested needs only a change in the *I2C* pin assignments. However, I am not sure where in the code I would update the pins. I have tried to avoid the error stated by the compiler by swapping out 'I2C i2c(I2C_SDA, I2C_SCL);' with 'MicroBitI2C i2c = MicroBitI2C(I2C_SDA0, I2C_SCL0);' , but I am not sure whether this is correct. If I do this swap, the compiler then considers the error solved and shows up with the next one:
- As provided and the swap from the last error implemented, the compiler states: "Error: Identifier "INT" is undefined in "main.cpp", Line: 139, Col: 16". The code uses the INT pin which isn't a specific pin on the microbit, so I'm not sure what that would connect to. Upon searching for how I would use an interrupt pin on a micro:bit, I found that it is possible to set a pin as the interrupt pin using (from [link](https://tinygo.org/docs/reference/microcontrollers/machine/microbit/#func-pin-setinterrupt)):

func (p Pin) SetInterrupt(change PinChange, callback func(Pin)) error

However, I am not sure how I would implement this into the micro:bit, or how it works.

Due to the uncertainty around the pulse oximeter, I sent an email to Maxim Integrated to ask them how I would go about fixing these errors. They then replied, suggesting that the best thing to do is to use the source code as a reference and develop on the make code micro:bit website. This, however, requires knowledge in C++, which none of us have. Due to this, to save time, we put the pulse oximeter on hold to get the rest of the project completed and are only now resuming.

Development

Whilst it was possible for me to code in JavaScript blocks, they did not have the flexibility of coding it yourself line-by-line and would have meant I was restricted to a very few objects I could use. In my code using the JavaScript editor, I often use 2 dimensional arrays and associative arrays and classes, all of which aren't possible in the blocks editor. If I had coded it in blocks, I would be severely limited to having data stored in lists and variables and the code being unreadable.

The first versions of the algorithm had the problem that they were written in python, which meant that the micro:bit ran on a JavaScript (TypeScript to be specific) version of the code and showed errors for that version of the code which I had not written.

```
accel_dict_raw[x] = input.acceleration(Dimension.Z)
```

Line 22: Element implicitly has an 'any' type because type '{}' has no index signature.

This problem confused me for some time, but after researching online about index signatures, I came to the conclusion that the problem actually originated in the JavaScript code and that it in fact was TypeScript because index signatures only exist in TypeScript. Following this, I had to switch to the JavaScript editor and added an 'any' type to each object I created (previously dictionaries in python). However, this sloppy use of types later meant that I got other errors like 'e.findIdx is not a function'.

This was specifically in regard to index signatures, which are only a thing in TypeScript and so to avoid this error, I continued using TypeScript for the algorithm, but this time included proper index signatures. This was first implemented in Version 2 of the code:

```
interface Raw_accel_array {
    [key: number]: number[];
}
```

This did however mean, I had to learn JavaScript. As I already knew python, it didn't take too long to acquaintance myself to the conventions of JavaScript. However, TypeScript was something completely new; I had never worked with types so closely before. Using the micro:bit JavaScript editor and encountering errors also gave me a feel to what my code actually means for the micro:bit. Debugging code is a skill which improves problem solving and requires perseverance, a simple syntax error is easy to miss but the more you do it, familiarising yourself with the various errors, the better at it you become.

The early versions of the algorithm were inaccurate as they would, in an attempt to conserve memory, reset at every 5th interval unless the accelerometer stated that there was a period of high acceleration. The following versions were a remake of this code, being cleaner, and more accurate at measuring falls.

Before it would work, as I had removed the auto-deleting function at every 5th interval, I had to ensure that the arrays in use were reset before they became too large and caused a memory error on the micro:bit. To solve the problem, I had to ensure that the arrays were only being added to if there has been a period of high acceleration as this would be when the sensor micro:bit then looks for stationary periods and ones in which the person is moving. If the person has a high acceleration, then stays at low (with a few anomalous times when the person is moving), then

a fall has been detected. If the person does not have a high acceleration, there is no point in adding that to the array and taking up more memory as the person cannot have fallen without a period of high acceleration. If the person starts continually moving after a fall, then the person must have not fallen, and all arrays are reset.

Version 2 marked a fully working version of the code which involved 2 micro:bits which interface with each other using radio and outputted to TeraTerm, serial software which allowed for both input and output to the receiver micro:bit.

Version 3 is also finished. This will allow for more in depth displaying of the data on a website which would use WebSerialAPI. The receiver micro:bit was previously 1-2kB over the file limit and so to reduce the size, I had to use an effective way of storing the data, and since strings take up lots of space, I had to shorten strings and provide abbreviations instead. The interpretations that Version 3 of the code makes is much more detailed and insightful compared to the fall or not fall output from Version 2.

The increased size of V3 of the sensor micro:bit code (previously ≈ 4kB over the file size limit) was combatted by not including any unnecessary functions. One that took up ≈ 3kB of space was:

```
basic.showIcon(IconNames.Target)
```

Initially it was included due to the increased simplicity as an LED screen didn't have to be drawn out. It was very unnecessary, but I had never suspected that this single line of code would be responsible for 3kB of the file size. Through thorough checking each function and procedure by commenting it out and checking if it takes up too much space, I determined it was this single line of code and replaced it with the usual alternative of drawing the LED screen. Before finding that this was the culprit, I decided to remove as many strings from my code as possible to reduce the file size whilst still preserving readability. I did this by creating variables that had the strings name and made their values numbers, which take up less space than text, cutting down on ≈ 200 bytes. I drew up a list of what to avoid ensuring a small file size which I had to test for checking which lines of code take up the most space:

- Do not use objects unnecessarily (including arrays and classes)
- Do not use the LED screen unnecessarily
- Do not use the inbuilt icons on LED screen (uses 3kB)
- Do not use led.plotAll() or similar functions (uses 70B)
- Always say `if(on == true)` instead of `if(on)` (uses 4B)

Block diagram

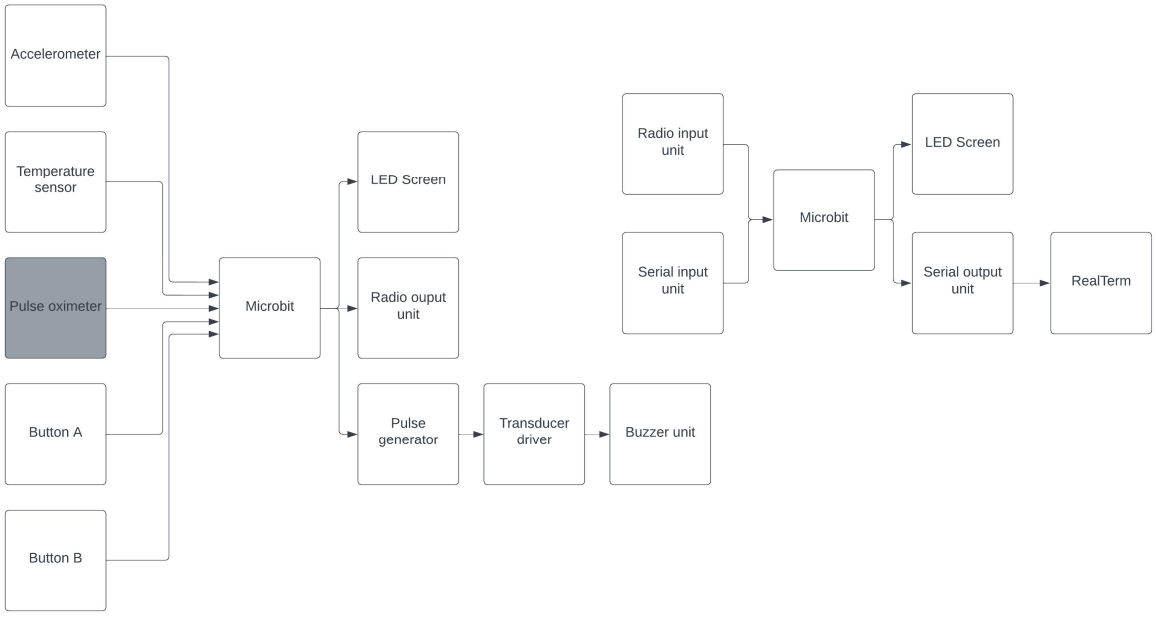


Figure 16 Block diagram (applies to both versions, but the temperature sensor is only V3, and pulse oximeter isn't a block currently being used)

Flowchart for V2 of sensor & receiver code

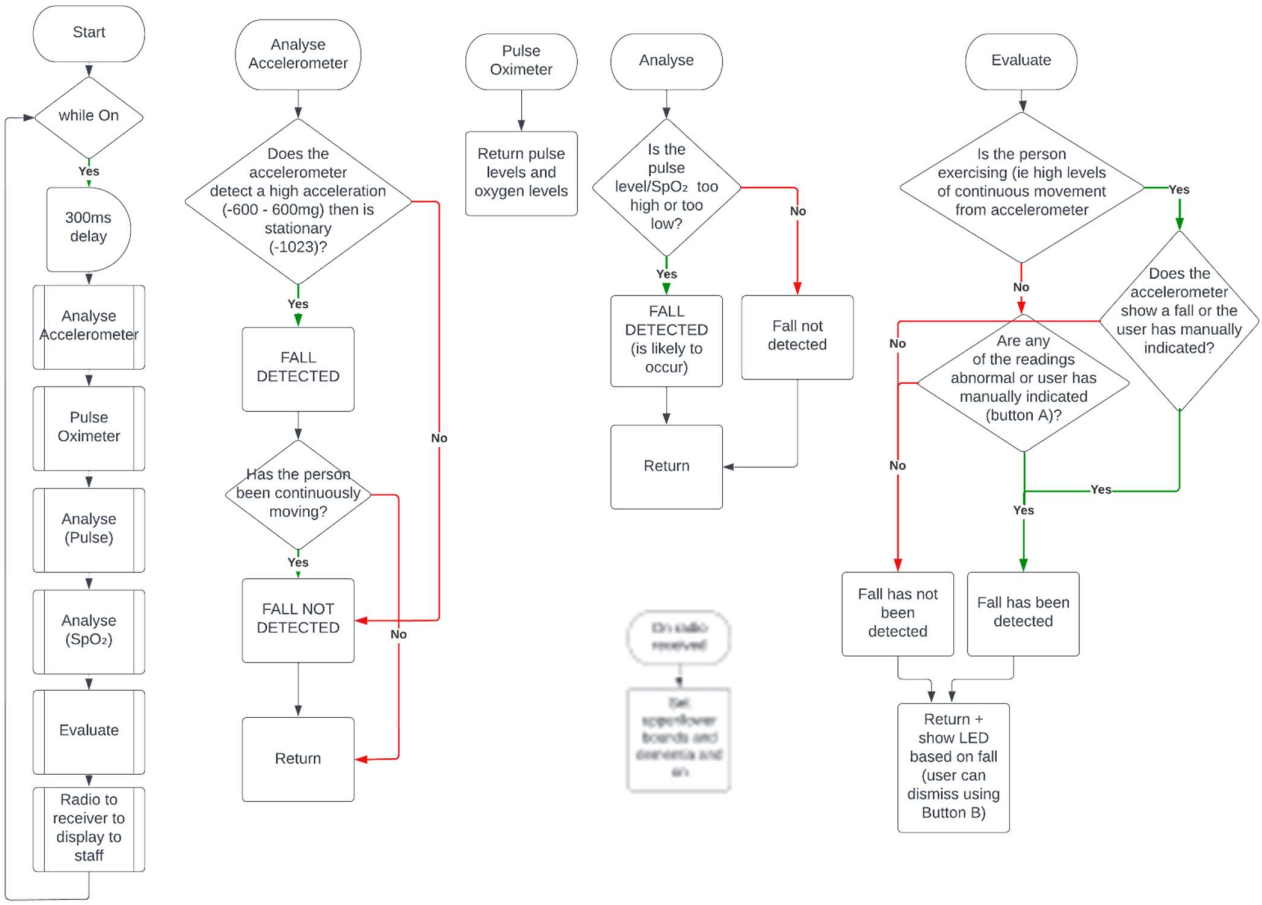
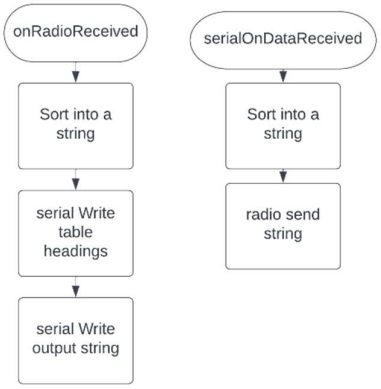


Figure 17 Flowchart for Version 2 of the Sensor micro:bit code - later versions expanded upon this flowchart and V3 moved the evaluate function to the receiver micro:bit



Setup instructions for receiver side for V2:

Install RealTerm

Set Display > Display As (left column) > Ansi

Set Port > Baud > 115200 (scroll up)

Set Send > EOL > +CR and +LF

Figure 18 Flowchart for the receiver micro:bit (code V2)

Flowchart for V3 of sensor code

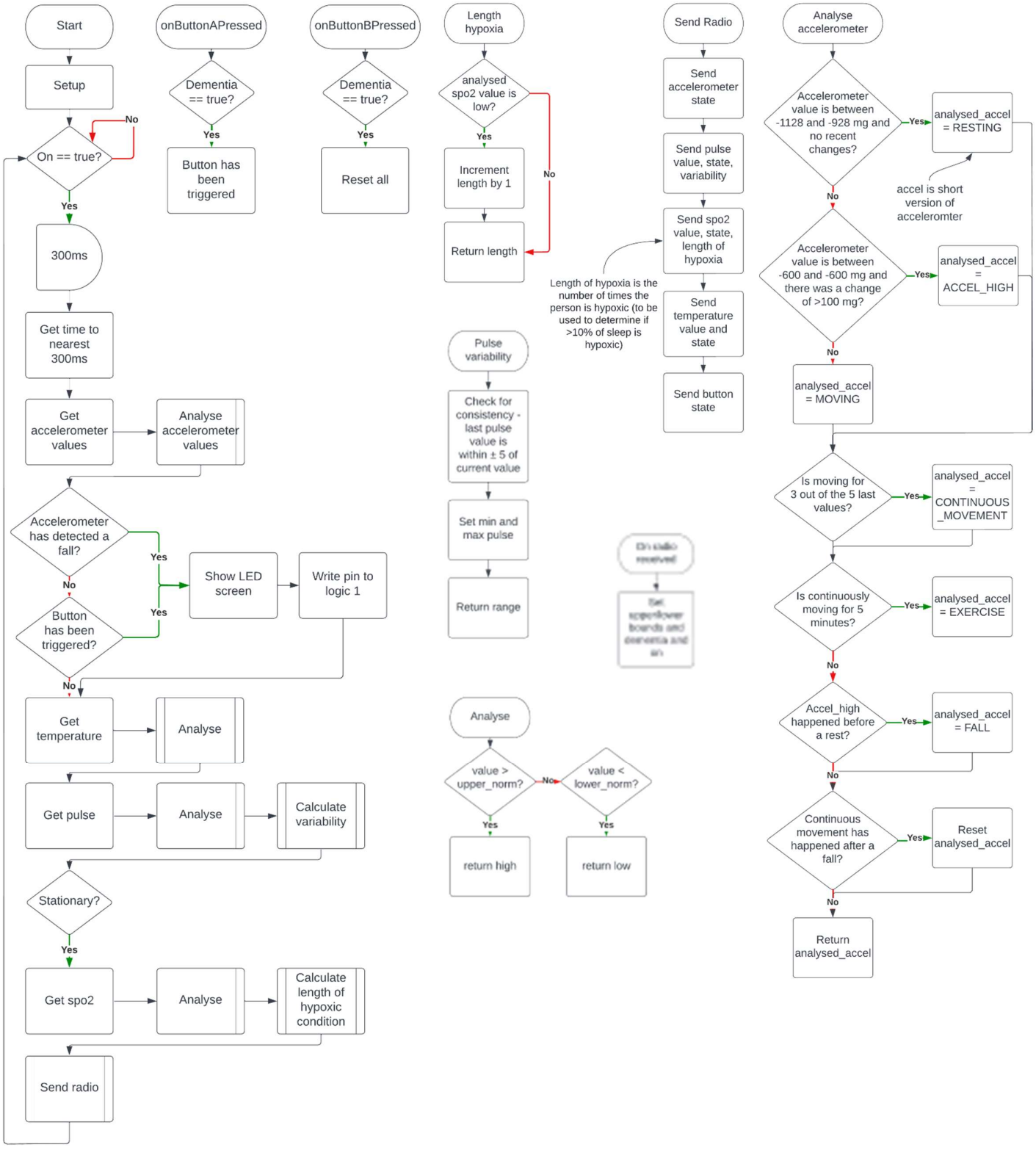


Figure 19 Flowchart for the sensor micro:bit (code V3)

Flowchart for V3 of receiver code

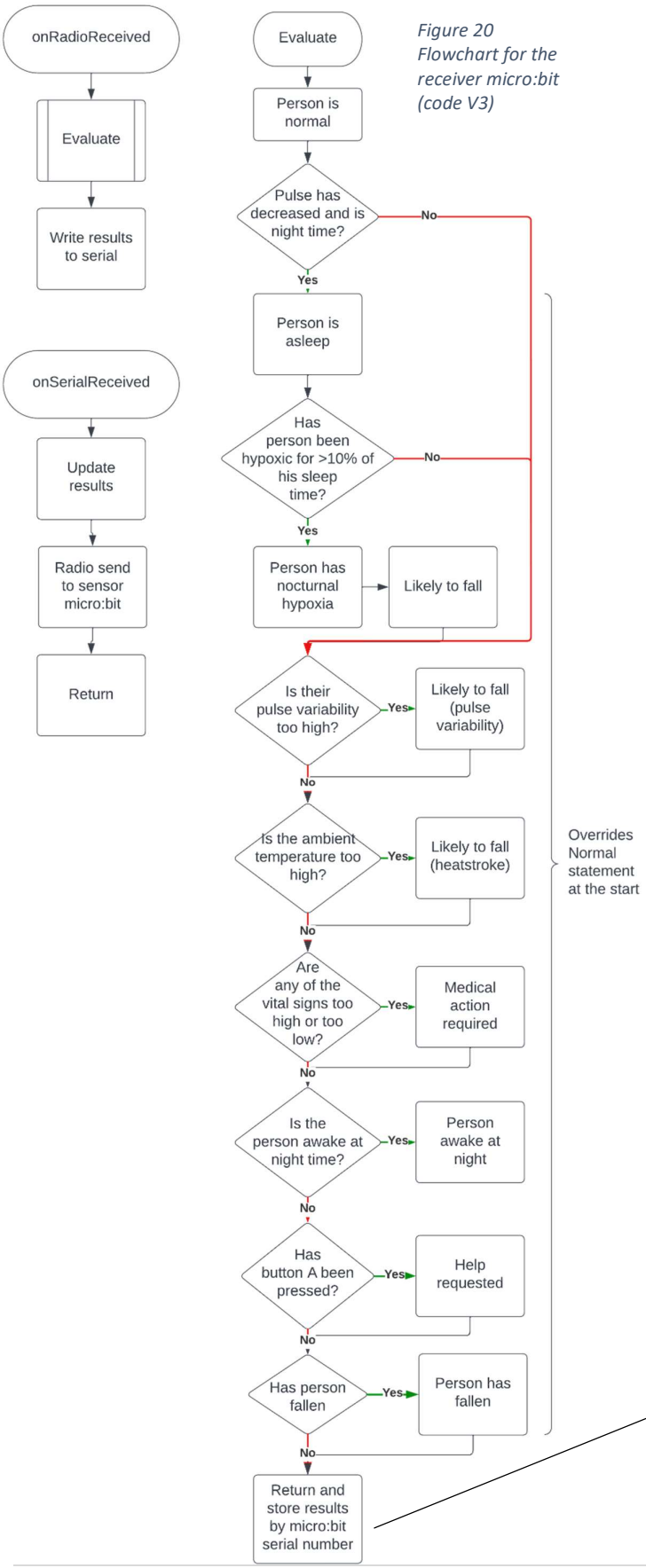


Figure 20 Flowchart for the receiver micro:bit (code V3)

Storage of the data processed by the flowchart is in the following format. The website which will use WebSerialAPI will then receive this data and unpick it, sorting it into the relevant table rows and for shortened strings (shortened to deal with the file size limit), they will be reverted to their proper form.

time

```
[6282,"",{"state":["Norm"],"accel":["rest"],
"pulse":["60"],"spo2":["95"],"temp":["21"],
"int":[""]},{"dementia":0,"U_pulse":60,"
L_pulse":100,"spo2":90,"on":1}]
```

Red: the data that staff input (if not inputted, set to default values) and are sent to the sensor micro:bit. They are also outputted via. serial in order to display on the input table.

Blue: the data that is outputted and sent to serial to display on the main output table

The above array is stored in a Class and when outputted, a class method is called to convert it to an array. The class as an object is stored itself by id in another array. The id is just the micro:bit serial number and this is what the flowchart shows with the output of the Evaluate function.

When adding a person, the id is required in order to create a space for that class and then the sensor micro:bit may be used to collect the required data

Storing the output of the evaluate function in a 'People_array' by 'id' under the 'output' property of the class

```
People_array[id].output = evaluate
(radio_array, id)
```

Version 2 output image

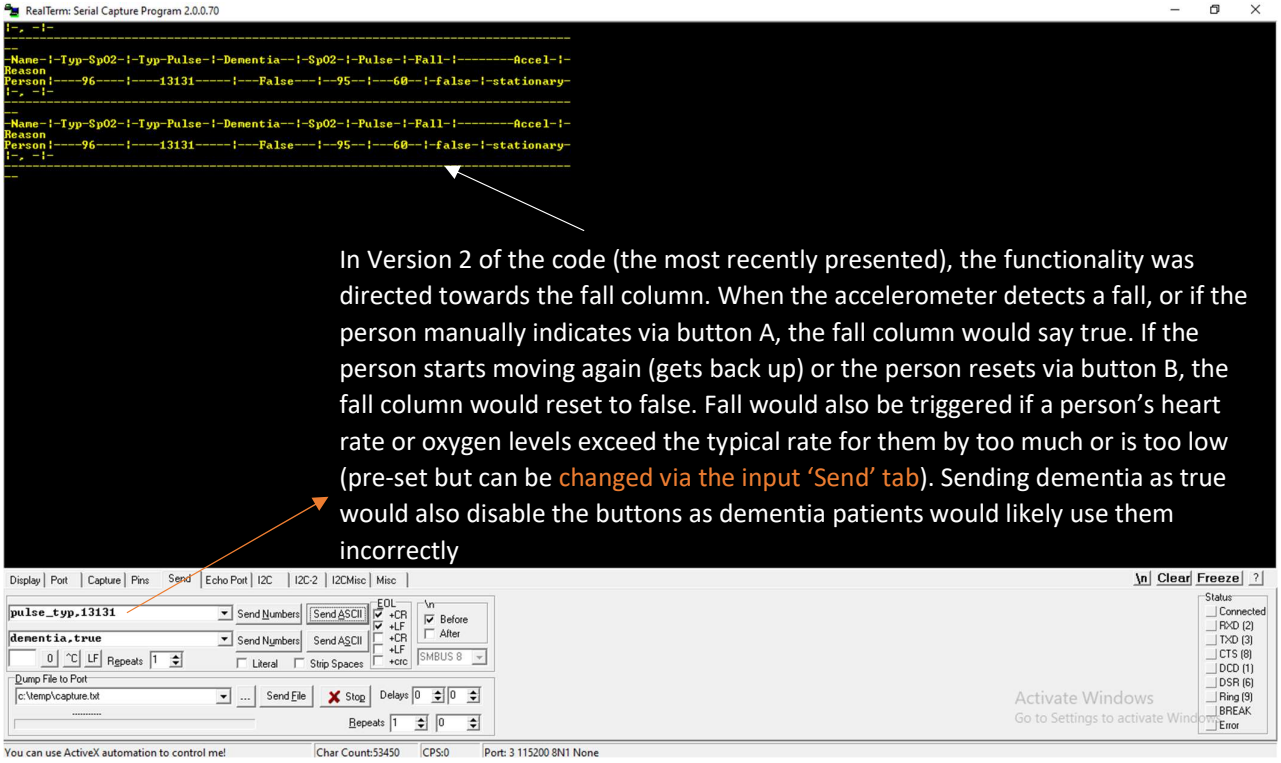


Figure 21 The output and input terminal of code V2, using RealTerm

The website

The website is how we would structure the data presentation in a care home environment as a product. We have made progress on the website, however, are still in the process of coding it. It depends on the code in version 3 to run, which needs optimisation to reduce file size. Optimisation has already been completed for the receiver micro:bit code, and a small amount of optimisation is needed for the sensor micro:bit.

The website's purpose is to provide a table of values recorded by the watch, which tells care home workers if users are going to fall or have fallen. Using this website, care home workers should be able to assist those about to fall or those who have fallen as soon as possible. This means that the website must:

- Be very easy to understand, and simple. For example, a simple table with a clear column about falls, and other columns about health-related data collected by the watch will be optimal. The reason for this is that if the website is too complex, workers may misinterpret the data being displayed, or may not be able to understand how to use the website at all, which could mean they take much longer to identify that a fall has occurred.
- Have a table that updates its data as quickly and as often as possible. This process can be automated using a simple script. It is important to update this table because the faster the care home worker receives information from the table, the faster they can respond to a fall.
- Alert care home workers about falls and other important data as quickly as possible. It is highly unlikely that care home workers will be able to monitor this website constantly, which means the website must be able to alert care home workers about important information when they are not looking at it. It can do this by playing sounds, using an alarm system, when important information is put into the table. This would allow workers to quickly identify that there is a situation that needs to be managed, even when they are not watching the website update. As a result of this, all falls and important data will be reported and attended to, and nothing can be overlooked, or missed.

- Not record any personal data. The only data that should be recorded (and should be able to be wiped instantly) is the current blood oxygen saturation, accelerometer state, temperature as well as the data that the staff input. We have ensured that in our layout we have included a way to delete people from the table to preserve people's privacy due to the Right to be forgotten. A key problem with many fall detection devices, and health-related devices as a whole is the privacy infringement issue, as these devices often track and record data of the user without the user's permission or knowledge. We have strongly decided against this, because we want to maintain the highest level of privacy for our users, in order to ensure that they are receiving the best possible care and protection without infringement of privacy.
    - Right to be forgotten: 'An individual has the right to have their personal data erased if: The personal data is no longer necessary for the purpose an organization originally collected or processed it.' (Wolford, 2020)

When designing the website, there were quite a few initial problems. To begin with, I had no initial way of making the website automatically refresh, which was a key part in the specification I had written beforehand. In order to solve this, I used my knowledge about functions in html, alongside extensive research into the most optimal way of automatically refreshing websites, to find the script shown in Figure 9. Finding the interval for this refresh was also an issue, as we needed to test the usage of the device to identify a suitable interval. In order to test this, we decided to use the device in a practical situation and calculated how often data was being inputted. From this, I was able to set the interval.

```
<script>
    function autoRefresh() {
        window.location = window.location.href;
    }
    setInterval('autoRefresh()', 5000);
</script>
```

Figure 22 The script used to automatically refresh the table with new data from the device.

Layout

+	Person	State *(click to dismiss)*	SpO_2	Pulse	Accelerometer	Temperature	Fallarm interpretation
✕	Person9	FALL	98	67	FALLEN	50	SUN-STROKE
✕	Person6	LIKELY TO FALL	100	90	stationary	30	20% change in pulse
✕	Person1	LIKELY TO FALL	86	70	moving	30	Nocturnal Hypoxia
✕	Person5	ACTION NEEDED	70	90	stationary	30	LOW BLOOD OXYGEN
✕	Person3	Awake at night	99	80	moving	30	Moving at night unsupervised
✕	Person2	Help requested	97	77	moving	30	The user has requested help via. button
✕	Person4	Normal	99	80	moving	30	-
	etc						

Figure 23 Table for data from receiver microbit

Person10	Exit
ID NUMBER	[the microbit serial number]
Does this person have dementia?	False
Lower bound for SpO_2	Default - 80
Upper bound for pulse rate	Default - 100
Lower bound for pulse rate	Default - 60
Extra medical information	. . .

Figure 24 The table for values that care home staff input

Implementation into a care-home setting

Our product would fundamentally need the following:

- Temperature sensor (can be done with a potential divider)
- Pulse oximeter
- Accelerometer
- Wi-Fi to app unit
- Output unit – buzzer
    - We have already detailed how this would look like as hardware components in the Astable build

- As one of the initial ideas, it could also include an RFID tag reader to detect when the watch and bottle become near, i.e., the person is drinking. This would also inform of hydration levels

Our device uses an accelerometer to detect if a person has fallen over and notify staff after a brief period, allowing quick response to the fall, and therefore less injury to the elderly patient. The heart rate monitor helps to notify staff when a fall may occur, due to older adults who have suffered from a fall being twice as likely to suffer from an irregular heartbeat known as atrial fibrillation, allowing for an even quicker response to falls or injuries. The heart rate monitor will also be useful in several other situations involving the health of the elderly person; however, we are focusing on it being used to detect and prevent falls. Furthermore, studies show that heart rate can vary by up to 20% during falls, meaning that the sensors will allow for an even more accurate prediction of when a fall may occur. A blood oxygen level sensor is included as well, as it can also be a symptom if a patient is about to fall, with hypoxia (when a person's blood oxygen level falls below a certain threshold) causing a reduction in physical capabilities, and by extensions falls which may lead to severe or life-threatening injuries to the elderly patient. The blood oxygen and heart rate sensors are also both part of the same component (a pulse oximeter), allowing for a more compact design and more comfort for the elderly patient, whilst still maintaining the required function for the system.

Our design also involves a manual override button, allowing the elderly person to easily contact for help if they need it, meaning that even if the fall is not detected by any of the sensors, the elderly person still has a way to reach out for help. The program that runs on the Micro:bit detects if a person has had a fall (detected by a high acceleration followed by a period of rest) and will reset if the person begins moving again. In a care home setting, the program would operate on each watch and interface with one receiver Micro:bit which will display the information (in our case this is done through the Micro:bit serial and radio communication), this allows for quick communication of statistics and therefore a quick response from the care home workers. We have used a 3D printed case for our design, enabling for a strong case, whilst also being adaptable to our needs and design, allowing for versatility whilst maintaining the function of the device.

Evaluation of project

As elderly people want independence, having staff to monitor them at all times can annoying. By having an automatic monitor that detects if they are likely to fall or have already fallen, the need for many staff all monitoring residents is reduced. Although our prototype is quite uncomfortable, with a pulse oximeter constantly attached to your wrist, it gives care home residents more autonomy over their day.

For our project we have successfully implemented an accelerometer to detect when a fall has occurred, we have also created an input and output terminal, which allows variation for boundaries used to detect when a fall has occurred, i.e., blood oxygen levels and heart rate may vary from person to person and using a variable boundary system can allow for this adaptability. We have also created and printed a 3D model of our design using a 3D printer and have ensured the comfort of the user whilst wearing the device. Our group has worked on implementing the pulse oximeter, and whilst not fully finished, we are in the process of developing this aspect of the device. Furthermore, as an additional feature, we used a buzzer involving a 555 clock configured as an astable to further enhance the design and alert the care home workers as to when a fall has occurred.

Our product is largely successful and achieves most of our specification points. However, when tested, it does not measure pulse rate or blood oxygen levels as the only sensor viable for use with the micro:bit has provided code that does not run on the micro:bit. We also have failed to produce logos for the case to allow people to locate which button does what. All other specification points are reached, and we have managed to combat the problem of falls and also address the fact that residents with dementia would not be able to send inputs like a button press whilst still giving residents a way to reach out for help (if they don't have dementia). Use of a pulse oximeter in our product would also combat the problem of medical help. In the 3rd version of the code, a temperature sensor is also included to measure ambient temperature and see if it is a stroke that has occurred/is likely to occur. In version 3 of the code, we have successfully managed to give a comprehensive interpretation of the data for the care home staff to act quickly.

The pulse oximeter not working with our micro:bit directed our attention to the research into microcontrollers that we did, realising that it would have saved much time if we had chosen a microcontroller before starting the realisation of the project, as after writing 600 lines of code in one language for the micro:bit, you will not be able to swap the board you are using. In future, before diving into the project, a more systematic approach is required where you analyse the problems and your theorised solutions and choose a microcontroller best for addressing those solutions, for example an Arduino, which would be a good choice for a project involving many sensors as they have many analog in pins (they have an analog to digital converter – ADC) and there is a wide market for sensors compatible with an Arduino.

Out of our project objectives, we have successfully completed all of them. Even if we haven't implemented the pulse oximeter, the rest of the project successfully completes our aim of proving that low-cost technology can help adult social care in a care-home setting. As well as implementing the pulse oximeter sensor, we are looking to further improve our project by having a website in which we can set our own formatting, and display data in a table rather than line-by-line, which, whilst it does display the data to the staff in a somewhat table-like format, it could be improved. Development on this started when in the evaluation it was recognised that the data was somewhat hard to read, and we have already started planning this second phase to our project, making sure to include the previous pulse oximeter stage which we had put on hold for some time in the interest of time management, and ensuring that we double check the peripherals we intend to use before buying them.

Bibliography

Alzheimer's Society. (2022, June 30). *Facts for the media*. Retrieved from Alzheimer's Society: https://www.alzheimers.org.uk/about-us/news-and-media/facts-media

Cauley, J. A.-I. (2014, October 4). *Journal of the American Geriatrics Society*. Retrieved from Hypoxia during sleep and the risk of falls and fractures in older men: the Osteoporotic Fractures in Men Sleep Study.: https://doi.org/10.1111/jgs.13069

Chow H, & Yang C. (2020, April 28). *JMIR Mhealth Uhealth*. Retrieved from Accuracy of Optical Heart Rate Sensing Technology in Wearable Fitness Trackers for Young and Older Adults: Validation and Comparison Study: https://mhealth.jmir.org/2020/4/e14707/

Doyle, K. (2015, March 13). *Reuters Health*. Retrieved from Falls may be tied to irregular heartbeat: https://www.reuters.com/article/us-health-falls-atrial-fibrillation-idINKBN0M91LC20150313#:~:text=(Reuters%20Health)%20%2D%20Older%20adults,population%20before%2C%E2%80%9D%20said%20Dr.

Freilich, S., & Barker, R. (2009). *BJMP.org*. Retrieved from Predicting falls risk in patients – the value of cardiovascular variability assessment: https://www.bjmp.org/content/predicting-falls-risk-patients-value-cardiovascular-variability-assessment

Healthline. (2017, January 10). *Healthline*. Retrieved from Hot and Cold: Extreme Temperature Safety: https://www.healthline.com/health/extreme-temperature-safety

How Equipment Works. (n.d.). *How Equipment Works*. Retrieved from How pulse oximeters work explained simply: https://www.howequipmentworks.com/pulse_oximeter/

Johns Hopkins Medicine. (2022, June 14). *Johns Hopkins Medicine*. Retrieved from Vital Signs (Body Temperature, Pulse Rate, Respiration Rate, Blood Pressure): https://www.hopkinsmedicine.org/health/conditions-and-diseases/vital-signs-body-temperature-pulse-rate-respiration-rate-blood-pressure

Kraudel, R. (2021, September 9). *Valencell*. Retrieved from Optical Heart Rate Monitoring: What You Need to Know: https://valencell.com/blog/optical-heart-rate-monitoring-what-you-need-to-know/

Mitton, A. (2021, November 22). *khealth*. Retrieved from Can Dehydration Cause High Blood Pressure?: https://khealth.com/learn/hypertension/can-dehydration-cause-high-blood-pressure/#:~:text=Therefore%2C%20when%20you're%20dehydrated,leading%20to%20low%20blood%20pressure

Munn, E., & Thomas, G. (2021, December 7). *Expert reviews*. Retrieved from Are pulse oximeters on fitness trackers reliable? The best wearables for measuring blood oxygen saturation: https://www.expertreviews.co.uk/fitness-trackers/1411960/best-pulse-oximeters--wearables-fitness-trackers

NHS inform. (2021, September 16). *NHS inform*. Retrieved from Falls and dementia: https://www.nhsinform.scot/healthy-living/preventing-falls/falls-and-dementia

Palladino, V. (2017, April 2). *Ars Technica*. Retrieved from How wearable heart-rate monitors work, and which is best for you: https://arstechnica.com/gadgets/2017/04/how-wearable-heart-rate-monitors-work-and-which-is-best-for-you/

Phelan, E. A. (2015). *The Medical clinics of North America*. Retrieved from Assessment and Management of Fall Risk in Primary Care Settings: https://doi.org/10.1016/j.mcna.2014.11.004

Smith, S. (2018, June 25). *Vital Record*. Retrieved from 10 Common Elderly Health Issues: https://vitalrecord.tamhsc.edu/10-common-elderly-health-issues/#:~:text=Every%2015%20seconds,osteoporosis%20and%20osteoarthritis

Wikipedia. (n.d.). *Wikipedia*. Retrieved from List of Arduino boards and compatible systems: https://en.wikipedia.org/wiki/List_of_Arduino_boards_and_compatible_systems

Wikipedia. (n.d.). *Wikipedia*. Retrieved from Raspberry Pi: https://en.wikipedia.org/wiki/Raspberry_Pi

Wolford, B. (2020, April 24). *GDPR.EU*. Retrieved from Right to be forgotten: https://gdpr.eu/right-to-be-forgotten/

World Health Organization. (2021, April 26). *Falls*. Retrieved from World Health Organization: https://www.who.int/news-room/fact-sheets/detail/falls

Appendix A: Sensor Code V2

This algorithm is the one used in the most recent tests. It does some basic interpretation and also includes the evaluation function within the sensor code, something that we moved to the receiver in Version 3

```
let reason = ""
let button = false
let pulseo2: number[] = []
let time = 0
let test_spo2 = 0
let test_pulse = 0
let fall_detected = false
let accel_high: boolean = false
let count = 0
class Sensor_setup {
    lower_norm: number; upper_norm: number; stationary: number; scope: number; upper_stationary: number; lower_stationary: number; movement_reset: number;
    constructor(lower_norm: number, upper_norm: number, stationary: number, scope: number, upper_stationary: number, lower_stationary: number, movement_reset: number) {
        this.lower_norm = lower_norm; this.upper_norm = upper_norm; this.stationary = stationary; this.scope = scope, this.upper_stationary = upper_stationary, this.lower_stationary = lower_stationary, this.movement_reset = movement_reset
    }
}
const accel_propty = new Sensor_setup(-600, 600, -1023, 100, -923, -1123, 3)
const pulse_propty = new Sensor_setup(60, 100, null, null, null, null, null)
const spo2_propty = new Sensor_setup(95, 100, null, null, null, null, null)
let interval = 300
let on = true

////////////////
test_pulse = 60
test_spo2 = 95
////////////////
let reset = false
interface Raw_accel_array {
    [key: number]: number[];
}
interface Raw_arrays {
    [key: number]: number;
}
interface Analysed_arrays {
    [key: number]: string
}
let raw_accel_array: Raw_accel_array = {}
let last_values = [input.acceleration(Dimension.X), input.acceleration(Dimension.Y), input.acceleration(Dimension.Z)]
let last_analysed = ["0", "0", "0"]
let analysed_accel_array: Analysed_arrays = {}
let raw_pulse_array: Raw_arrays = {}
let analysed_pulse_array: Analysed_arrays = {}
let raw_spo2_array: Raw_arrays = {}
let analysed_spo2_array: Analysed_arrays = {}
let accel_high_index: number
let stationary_index: number
let continuous_movement_index: number

let accel_timestamp = 0
let moving = false
let stationary = false

let dementia = false


function size(array: Analysed_arrays | Raw_arrays | Raw_accel_array) {
    let size = 0
    let my_keys = Object.keys(array)
    size = my_keys.length
    return size
}

```

```
62. function indexOf(array: any, item: any, time: number) {
63.     let index = -10000
64.     for (let i = 0; i < (time + interval); i += interval) {
65.         if (array[i] == item) {
66.             index = i
67.         }
68.     }
69.     return index
70. }
71.
72. function in_array_counter(array: string[], item: any) {
73.     let moving_count = 0
74.     for (let j = -1; j < (array.length); j++) {
75.         if (array[j] == item) {
76.             moving_count++
77.         }
78.     }
79.     return moving_count
80. }
81.
82. function str_to_bool(str: string) {
83.     if (str == "true") {
84.         return true
85.     } else {
86.         return false
87.     }
88. }
89.
90. function resetter(extent: string) {
91.     if (extent == 'accel' || extent == 'all') {
92.         analysed_accel_array = {}
93.         analysed_accel_array[time] = 'Normal'
94.         accel_high = false
95.         raw_accel_array = {}
96.     }
97.     if (extent == 'pulse' || extent == 'all') {
98.         analysed_pulse_array = {}
99.         raw_pulse_array = {}
100.      }
101.      if (extent == 'spo2' || extent == 'all') {
102.          analysed_spo2_array = {}
103.          raw_spo2_array = {}
104.      }
105.      if (extent == 'button' || extent == 'all') {
106.          button = false
107.      }
108.  }
109.
110.  function get_radio() {
111.      return
112.      /*
113.      - No physical clock, however, the input of time is required to find if a person is moving at night
    time - can be helped to get to the toilet or wherever they need to go
114.      - Hypoxia during sleep can be detected with the clock as well, determining if there is a fall.
115.      - Max and Min pulse and spo2 levels should be inputted, if not inputted then it is set to the
    default.*/
116.
117.      //if (str == 'reset'){control.reset()} - allows staff to reset microbit
118.      //if (str == 'off){on = false}
119.  }
120.
121.
122.  function f_to_radio(time: number, reason: string, analysed_accel_array: Analysed_arrays,
    raw_pulse_array: Raw_arrays, raw_spo2_array: Raw_arrays, button: boolean, fall_detected: boolean) {
123.
124.      radio.setGroup(101)
125.      radio.setTransmitSerialNumber(true)
126.
127.      //radio.sendString("time" + "," + time)
128.      radio.sendString("fall" + "," + fall_detected.toString())
129.      radio.sendString("reason" + "," + reason)
130.      if (moving == false && stationary == false) {
131.          radio.sendString("accel" + "," + analysed_accel_array[time])
```

```
132.      } else if (stationary == true) {
133.          radio.sendString("accel" + "," + 'stationary')
134.      } else if (moving == true) {
135.          radio.sendString("accel" + "," + 'moving')
136.      }
137.
138.      radio.sendString("pulse" + "," + raw_pulse_array[time].toString())
139.      radio.sendString("spo2" + "," + raw_spo2_array[time].toString())
140.
141.  }
142.
143.
144.
145.  function f_pulseo2(time: number) {
146.      // get data from sensor, break into two, then store in arrays
147.      let raw_pulse_value = test_pulse
148.      let raw_spo2_value = test_spo2
149.      return [raw_pulse_value, raw_spo2_value]
150.  }
151.
152.
153.
154.  function f_analyse_accel(analysed_array: Analysed_arrays, array: Raw_accel_array, time: number) {
    //exclusively for accelerometer analysis
155.      //array passed follows this format: {time: [X,Y,Z], time: [X,Y,Z] }
156.
157.      let x = 0, y = 1, z = 2
158.      if (count == 0) {
159.          basic.clearScreen()
160.      }
161.
162.      moving = false
163.      stationary = false
164.
165.      //stationary-----------------------------------------------------------------------------------
    ---
166.      if (
167.          ((accel_propty.lower_stationary < array[time][z] && array[time][z] <
    accel_propty.upper_stationary)
168.              || (Math.abs(array[time][z] - last_values[z]) < accel_propty.scope))
169.          && (Math.abs(array[time][x] - last_values[x]) < accel_propty.scope)
170.          && (Math.abs(array[time][y] - last_values[y]) < accel_propty.scope)
171.      ) {
172.          //led.plot(count % 5, 2)
173.
174.          if (accel_high) {
175.              analysed_array[time] = 'stationary'
176.              stationary_index = indexOf(analysed_array, 'stationary', time)
177.              /*if (in_array_counter(last_analysed, 'stationary') > accel_propty.movement_reset) {
178.                  reset = true
179.              }*/
180.          }
181.          stationary = true
182.      }
183.
184.      //accel_high-----------------------------------------------------------------------------------
    ---
185.      else if (
186.          (accel_propty.lower_norm < array[time][z] && array[time][z] < accel_propty.upper_norm)
187.          && array[time][z] - last_values[z] > accel_propty.scope
188.      ) {
189.          //led.plot(count % 5, 0), //led.plot(count % 5, 4)
190.          analysed_array[time] = 'accel_high'
191.          accel_high = true
192.          accel_timestamp = time
193.          accel_high_index = indexOf(analysed_array, 'accel_high', time)
194.      }
195.
196.      //ordinary movement----------------------------------------------------------------------------
    --
197.      else {
198.          if (accel_high) {
199.              analysed_array[time] = 'moving'
```

```
        }
        moving = true
        //led.plot(count % 5, 1), //led.plot(count % 5, 3)
    }

    //continuous_movement-------------------------------------------------------------------------------
    if (in_array_counter(last_analysed, 'moving') >= accel_propty.movement_reset) {
        if (accel_high) {
            analysed_array[time] = 'continuous_movement'
        }

        if (accel_high){
            if (time - accel_timestamp > 4000){
                accel_high = false
                accel_high_index = -10000
            }
        }
        //led.plot(count % 5, 1), //led.plot(count % 5, 2), //led.plot(count % 5, 3)
    }

    //exercise-----------------------------------------------------------------------------------------
    if (in_array_counter(last_analysed, 'continuous_movement') >= 300000 / interval) {
        analysed_array = {}
        analysed_array[time] = 'exercise'
        accel_high = false
        //led.plot(count % 5, 1), //led.plot(count % 5, 2), //led.plot(count % 5, 3)

        //will ignore any anomalous pulse readings or spo2 readings
    }



    //FALLS--------------------------------------------------------------------------------------------
    if (analysed_array[time] == 'stationary' && accel_high == true && accel_high_index != -10000) {
            if (accel_high_index < stationary_index) {
                analysed_array[time] = 'fall'
                basic.clearScreen()

            }

            if (accel_high_index == -10000) { reset = true }

    }

    last_values = [array[time][x], array[time][y], array[time][z]]
    count += 1
    last_analysed[count % (accel_propty.movement_reset + 1)] = analysed_array[time]  //store the last 5 analysed_accel_array values
    if (count % 5 == 0 && fall_detected == false) { basic.clearScreen() } //when to clear screen for next LED sequence to occur

    if (reset || (accel_high == false)) {
        resetter('accel')
    }

    return analysed_array
}


function f_analyse(analysed_array: Analysed_arrays, array: Raw_arrays, time: number, upper_norm: number, lower_norm: number) {
    analysed_array[time] = 'normal'
    if (array[time] > upper_norm) {
        analysed_array[time] = 'high'
        basic.showLeds(`
        . . # . .
        . # # # .
        # . # . #
        . . # . .
        . . # . .
        `)
    }
```

```
269.
270.      if (array[time] < lower_norm) {
271.          analysed_array[time] = 'low'
272.          basic.showLeds(`
273.          . . # . .
274.          . . # . .
275.          # . # . #
276.          . # # # .
277.          . . # . .
278.          `)
279.      }
280.      return analysed_array
281.  }
282.  function f_evaluate(analysed_accel_array: Analysed_arrays, analysed_pulse_array: Analysed_arrays,
    analysed_spo2_array: Analysed_arrays, button: boolean, time: number) {
283.      let fall_detected_formatted = false
284.      reason = ''
285.      if ((analysed_accel_array[time] == 'fall' || analysed_pulse_array[time] == 'high' ||
    analysed_spo2_array[time] == 'high' || button) && analysed_accel_array[time] != 'exercise') {
286.          fall_detected_formatted = true
287.          basic.showLeds(`
288.              # # # # #
289.              # # # # #
290.              # # # # #
291.              # # # # #
292.              # # # # #
293.              `)
294.
295.          if (analysed_accel_array[time] == "fall") {
296.              reason += "Movement;"
297.          }
298.          if (analysed_pulse_array[time] == "high") {
299.              reason += "Pulse;"
300.          }
301.          if (analysed_spo2_array[time] == "high") {
302.              reason += "Oxygen;"
303.          }
304.          if (button) {
305.              reason += "User;"
306.          }
307.      }
308.      if (analysed_accel_array[time] == 'exercise' && analysed_accel_array[time] == 'fall') {
309.          //Fall detected cannot be triggered by pulse and spo2
310.          if (analysed_accel_array[time] == "fall") {
311.              reason += "Movement;"
312.          }
313.          if (button) {
314.              reason += "User;"
315.          }
316.      }
317.      let my_string = '' + fall_detected_formatted
318.      let my_list: any = [my_string, reason]
319.      return my_list
320.  }
321.
322.
323.  // MAIN flow
324.  loops.everyInterval(interval, function () {
325.      if (on) {
326.          pins.digitalWritePin(DigitalPin.P0, 0)
327.          //makes buzzer pulse if uncommented. If commented, astable circuit is required
328.          basic.pause(interval)
329.          time = Math.round(input.runningTime() / interval) * interval // rounds to nearest interval
330.          // {time: [X,Y,Z], time: [X,Y,Z] }
331.
332.          get_radio()
333.
334.          raw_accel_array[time] = [input.acceleration(Dimension.X), input.acceleration(Dimension.Y),
    input.acceleration(Dimension.Z)]
335.          analysed_accel_array = f_analyse_accel(analysed_accel_array, raw_accel_array, time)
336.
337.          pulseo2 = f_pulseo2(time)
338.          raw_pulse_array[time] = pulseo2[0]
```

```
339.          raw_spo2_array[time] = pulseo2[1]
340.
341.          analysed_pulse_array = f_analyse(analysed_pulse_array, raw_pulse_array, time,
    pulse_propty.upper_norm, pulse_propty.lower_norm)
342.          analysed_spo2_array = f_analyse(analysed_spo2_array, raw_spo2_array, time,
    spo2_propty.upper_norm, spo2_propty.lower_norm)
343.
344.          let fall_list: string[] = f_evaluate(analysed_accel_array, analysed_pulse_array,
    analysed_spo2_array, button, time)
345.          let fall_detected_string: string = fall_list[0]
346.          fall_detected = str_to_bool(fall_detected_string)
347.          reason = fall_list[1]
348.
349.          f_to_radio(time, reason, analysed_accel_array, raw_pulse_array, raw_spo2_array, button,
    fall_detected)
350.
351.          if (fall_detected) {
352.              pins.digitalWritePin(DigitalPin.P0, 1)
353.          } else {
354.              pins.digitalWritePin(DigitalPin.P0, 0)
355.          }
356.
357.          raw_pulse_array = {}
358.          analysed_pulse_array = {}
359.          raw_spo2_array = {}
360.          analysed_spo2_array = {}
361.          analysed_accel_array = {}
362.          raw_accel_array = {}
363.
364.
365.      }
366.
367.  })
368.
369.
370.  input.onButtonPressed(Button.A, function () {
371.      if (!dementia) { button = true }
372.      //led.plot(0, 0), //led.plot(4, 4)
373.  })
374.
375.  input.onButtonPressed(Button.B, function () {
376.      //led.plot(4, 0), //led.plot(0, 4)
377.      if (!dementia) { resetter('all') }
378.  })
379.
380.  radio.onReceivedString(function(receivedString: string) {
381.      let edit_string = receivedString.split(',')
382.      if (edit_string[0] == 'spo2_typ') {
383.          spo2_propty.upper_norm = parseInt(edit_string[1]) + 20
384.          spo2_propty.lower_norm = parseInt(edit_string[1]) - 20
385.      } else if (edit_string[0] == 'pulse_typ') {
386.          pulse_propty.upper_norm = parseInt(edit_string[1]) + 20
387.          pulse_propty.lower_norm = parseInt(edit_string[1]) - 20
388.      } else if (edit_string[0] == 'dementia') {
389.          if (edit_string[1] == 'true' || edit_string[1] == 'True') { dementia = true }
390.          if (edit_string[1] == 'false' || edit_string[1] == 'False') { dementia = false }
391.      }
392.  })
393.
```

Appendix B: Receiver Code V2

This algorithm is the one used for the most recent tests. It functions on the basis of radio communication between the two micro:bits and sends, via. the serial port, a table to be outputted to TeraTerm (serial software). This code was previously over the limit but has since been optimised to save space.

```
1.  radio.onReceivedString(function (receivedString) {
2.
3.      led.plot(2, 0)
4.      split_string = receivedString.split(",")
5.      id2 = radio.receivedPacket(RadioPacketProperty.SerialNumber)
6.      time = radio.receivedPacket(RadioPacketProperty.Time)
7.      if (split_string[0] == "fall") {
8.          fall = split_string[1]
9.      }
10.     if (split_string[0] == "reason") {
11.         let reason = split_string[1]
12.         let reasons_list = reason.split(';')
13.         reasons = ''
14.         for (let i of reasons_list) {
15.             if (i == 'Us') {
16.                 reasons = reasons + ', ' + 'User'
17.             } else {
18.                 reasons = reasons + ', ' + i
19.             }
20.
21.         }
22.     }
23.
24.     if (split_string[0] == "accel") {
25.         accel = split_string[1]
26.     }
27.     if (split_string[0] == "pulse") {
28.         pulse = split_string[1]
29.     }
30.     if (split_string[0] == "spo2") {
31.         spo2 = split_string[1]
32.     }
33.     if (fall == "true" && reasons != "None" && accel != "Not sent" && pulse != "Not sent" && spo2 != "No
    t sent") {
34.         serial.writeLine("-Name-|-Typ-SpO2-|-Typ-Pulse-|-Dementia--|-SpO2-|-Pulse-|-Fall-|--------Accel-
    |-Reason ")
35.         serial.writeLine("Person|----" + spo2_typ + "----|----" + pulse_typ + "-----|---
    " + dementia + "----|--" + spo2 + "--|---" + pulse + "--|-" + fall + "-|-" + accel + "-|-
    " + reasons + '-|-')
36.         //serial.writeLine("Person|----96----|----67-----|---True---\|--97--|---75--|-")
37.         //serial.writeLine("Person|----96----|----67-----|---False---|--97--|---75--|-")
38.
39.         serial.writeString('\n')
40.         serial.writeLine('-------------------------------------------------------------------------------
    ----')
41.         fall = "None"
42.         reasons = "None"
43.         accel = "Not sent"
44.         pulse = "Not sent"
45.         spo2 = "Not sent"
46.
47.         basic.showLeds(`
48.         # . . . #
49.         . # . # .
50.         . . # . .
51.         . # . # .
52.         # . . . #
53.         `)
54.     }
55.     if (fall == "false" && accel != "Not sent" && pulse != "Not sent" && spo2 != "Not sent") {
```

```
        serial.writeLine("-Name-|-Typ-SpO2-|-Typ-Pulse-|-Dementia--|-SpO2-|-Pulse-|-Fall-|--------Accel-")
        serial.writeLine("Person|----" + spo2_typ + "----|----" + pulse_typ + "-----|---" + dementia + "----|--" + spo2 + "--|---" + pulse + "--|-" + fall + "-|-" + accel + "-|-")
        serial.writeLine('-------------------------------------------------------------------------------------')
        fall = "None"
        reasons = "None"
        accel = "Not sent"
        pulse = "Not sent"
        spo2 = "Not sent"
    }

    basic.clearScreen()
})

serial.onDataReceived(serial.delimiters(Delimiters.NewLine), function () {
    led.plot(2, 4)
    input_string = serial.readUntil(serial.delimiters(Delimiters.NewLine)).split(',')
    if (input_string[0] == 'spo2_typ') {
        spo2_typ = parseInt(input_string[1])
        radio.sendString('spo2_typ, ' + input_string[1])
    } else if (input_string[0] == 'pulse_typ') {
        pulse_typ = parseInt(input_string[1])
        radio.sendString('pulse_typ, ' + input_string[1])
    } else if (input_string[0] == 'dementia') {
        dementia = input_string[1]
        radio.sendString('dementia, ' + input_string[1])
    }

})

let input_string: string[]
let reasons_list: string[] = []
let reason = ""
let time = 0
let id2 = 0
let split_string: string[] = []
let spo2_typ = 0
let pulse_typ = 0
let fall = "Not sent"
let reasons = "Not sent"
let accel = "Not sent"
let pulse = "Not sent"
let spo2 = "Not sent"
let dementia = 'false'

radio.setGroup(101)
pulse_typ = 80
spo2_typ = 96

let reference_time: number

```

Appendix C: Sensor Code V3

[see description in code snippet]. It is a much clearer format with a more advanced algorithm compared to V2.

```
1.  //-----------------------------------------------------------------------------------------------
2.  /*
3.   * Team: Fallarm
4.   * Project: Royal Society
5.   * Product: Fall prevention and detection (FPAD)
6.   * Version number: 3.1
7.   * File name: Fallarm-V3-Sensor
8.   * Description: Monitors accelerometer, pulse oximeter and temperature to inform of a potential fall
9.
10. List of shortened strings/var names and their full counterparts:
11.     a_hi    => accel_high
12.     move    => moving
13.     c_move  => continuous_moving
14.     rest    => stationary
15. */
16. //-----------------------------------------------------------------------------------------------
17.
18. // GENERAL SETUP
19.     let button = false
20.     let time = 0
21.     let count = 0
22.     let interval = 300
23.     let on = true
24.     let reset = false
25.     let dementia = false
26.     class Sensor_setup {
27.         lower_norm: number; upper_norm: number; rest: number; scope: number; movement_reset: number;
28.         constructor(lower_norm: number, upper_norm: number, rest: number, scope: number, movement_reset:
     number) {
29.             this.lower_norm = lower_norm; this.upper_norm = upper_norm; this.rest = rest; this.scope = s
    cope, this.movement_reset = movement_reset
30.         }
31.     }
32. //}
33.
34.
35. //INDEX SIGNATURES {
36.     interface Raw_accel_array {
37.         [key: number]: number[];
38.     }
39.     interface Raw_arrays {
40.         [key: number]: number;
41.     }
42.     interface Analysed_arrays {
43.         [key: number]: string
44.     }
45. //}
46.
47. // PULSE OXIMETER SETUP {
48.     let raw_pulse: number
49.     let analysed_pulse: string
50.     let raw_spo2: number
51.     let analysed_spo2: string
52.     //movement_reset for pulse_propty is the number of pulse values
53.     //that must be above the current pulse value for it to be considered in pulse variability calculatio
    ns
54.     //scope here is the threshold for pulse variability
55.     //rest here is the upper bound of sleeping (rest) pulse
56.     let pulse_propty = new Sensor_setup(60, 100, 60, 20, 2)
57.     let spo2_min = 90
58.     let spo2_max = 100
59.
60.     let max_pulse = 1000
61.     let min_pulse = 0
62.     let range_pulse = 0
63.     let pulse_variability: string
```

```
    let spo2_hypoxic_len = 0 //the number of times the person is hypoxic (to be used to determine if >10 % of sleep is hypoxic)

    let last_pulse: number[] = []
    ////////////////////////
    let test_pulse = 60 //
    let test_spo2 = 95 //
    ////////////////////
//}

// TEMPERATURE SETUP {
    //const temp_propty = new Sensor_setup(20, 40, null, null, null)
    const temp_max = 40
    const temp_min = 20
    let raw_temp: number
    let analysed_temp: string
//}

// ACCELEROMETER SETUP {
    let a_hi: boolean = false
    let accel_timestamp = 0
    let move = false
    let rest = false
    let accel_high_index: number
    let rest_index: number
    let c_move_index: number
    const accel_propty = new Sensor_setup(-600, 600, -1023, 100, 3)

    //optimisation variables {
        const NORM = 0
        const REST = 10
        const MOVING = 5
        const A_HIGH = 100
        const FALL = -100
        const C_MOVING = 55
        const EXERCISE = 555
    //}

     let raw_accel_array: Raw_accel_array = {}
     let last_values = [input.acceleration(Dimension.X), input.acceleration(Dimension.Y), input.acceleration(Dimension.Z)]
     let last_analysed: number[] = []
     let analysed_accel_array: Raw_arrays = {}
 //}

 //-------------------------------------------------------------------------------------------------

 // FUNCTIONS

 function indexOf(array: any, item: any, time: number) {
     let index = -100
     for (let i = 0; i < (time + interval); i += interval) {
         if (array[i] == item) {
             index = i
         }
     }
     return index
 }

 function in_array_counter(array: number[], value: number, scope: number) {
     let moving_count = 0
     for (let j = -1; j < (array.length); j++) {
         if (array[j] >= value - scope && array[j] <= value + scope) {
             moving_count++
         }
     }
     return moving_count
 }

 function resetter(all: boolean) {
     //all 0 means only reset accel, all 1 means all
     analysed_accel_array = {}
     analysed_accel_array[time] = NORM
```

```
135.      a_hi = false
136.      raw_accel_array = {}
137.      basic.clearScreen()
138.
139.      if (all == true) {
140.          button = false
141.      }
142.  }
143.
144.  function f_to_radio(time: number, analysed_accel_array: Raw_arrays, str_pulse: string, str_spo2: strin
    g, str_temp: string, button_press: boolean) {
145.      radio.setGroup(101)
146.      radio.setTransmitSerialNumber(true)
147.
148.      let acceleration: string
149.      if (move == false && rest == false) {
150.          acceleration = 'other'
151.      } else if (rest == true) {
152.          acceleration = 'rest'
153.      } else if (move == true) {
154.          acceleration = 'move'
155.      }
156.      if (a_hi == true) {
157.          acceleration = 'fall'
158.      }
159.
160.      radio.sendString("accel" + "," + acceleration + ';')
161.      radio.sendString("pulse" + "," + str_pulse)
162.      radio.sendString("spo2" + "," + str_spo2)
163.      radio.sendString("temp" + "," + str_temp)
164.      radio.sendString("button" + ',' + button_press.toString() + ';')
165.  }
166.
167.  function f_get_pulse() {
168.      // get data from sensor
169.      let raw_pulse_value = test_pulse
170.      return raw_pulse_value
171.  }
172.
173.  function f_get_spo2() {
174.      // get data from sensor
175.      let raw_spo2_value = test_spo2
176.      return raw_spo2_value
177.  }
178.
179.  function f_pulse_variability(time: number, raw_pulse: number) {
180.      //Calculates the variation of heart rate, if this gets to pulse_propty.scope, then a fall is likel
    y
181.      last_pulse[count % (pulse_propty.movement_reset + 1)] = raw_pulse
182.
183.      let pulse_variability: string
184.
185.      if (in_array_counter(last_pulse, raw_pulse,5) >= pulse_propty.movement_reset) {
186.          //check for consistency - must be of that particular value to be counted for the following cal
    culations
187.          if (raw_pulse > min_pulse) { min_pulse = raw_pulse }
188.          if (raw_pulse < max_pulse) { max_pulse = raw_pulse }
189.          range_pulse = max_pulse - min_pulse
190.      }
191.
192.      if (range_pulse > pulse_propty.scope) {
193.          pulse_variability = 'hi'
194.      } else {
195.          pulse_variability = 'lo'
196.      }
197.      return pulse_variability
198.  }
199.
200.  function length_hypoxia(spo2_analysed: string, length: number) {
201.      if (spo2_analysed == 'lo') {
202.          length++
203.      }
204.      return length
```

```
205.  }
206.
207.  function f_analyse_accel(analysed_array: Raw_arrays, array: Raw_accel_array, time: number) { //exclusi
    vely for accelerometer analysis
208.      //array passed follows this format: {time: [X,Y,Z], time: [X,Y,Z] }
209.      const x = 0, y = 1, z = 2
210.
211.      if (count == 0) {
212.          basic.clearScreen()
213.      }
214.
215.      move = false
216.      rest = false
217.
218.      //rest-------------------------------------------------------------------------------------------
219.      if (
220.          (((accel_propty.rest - accel_propty.scope) < array[time][z] && array[time][z] < (accel_propty.
    rest + accel_propty.scope))
221.              || (Math.abs(array[time][z] - last_values[z]) < accel_propty.scope))
222.          && (Math.abs(array[time][x] - last_values[x]) < accel_propty.scope)
223.          && (Math.abs(array[time][y] - last_values[y]) < accel_propty.scope)
224.      ) {
225.          //led.plot(count % 5, 2)
226.
227.          if (a_hi == true) {
228.              analysed_array[time] = REST
229.              rest_index = indexOf(analysed_array, REST, time)
230.          }
231.          rest = true
232.      }
233.
234.      //a_hi-------------------------------------------------------------------------------------------
235.      else if (
236.          (accel_propty.lower_norm < array[time][z] && array[time][z] < accel_propty.upper_norm)
237.          && array[time][z] - last_values[z] > accel_propty.scope
238.      ) {
239.          //led.plot(count % 5, 0), //led.plot(count % 5, 4)
240.          analysed_array[time] = MOVING
241.          a_hi = true
242.          accel_timestamp = time
243.          accel_high_index = indexOf(analysed_array, MOVING, time)
244.      }
245.
246.      //ordinary movement-------------------------------------------------------------------------------
    --
247.      else {
248.          if (a_hi == true) {
249.              analysed_array[time] = MOVING
250.          }
251.          move = true
252.          //led.plot(count % 5, 1), //led.plot(count % 5, 3)
253.      }
254.
255.      //continuous_move--------------------------------------------------------------------------------
256.      if (in_array_counter(last_analysed, MOVING,0) >= accel_propty.movement_reset) {
257.          if (a_hi == true) {
258.              analysed_array[time] = C_MOVING
259.          }
260.
261.          if (a_hi == true) {
262.              if (time - accel_timestamp > accel_propty.movement_reset * 1000) {
263.                  a_hi = false
264.                  accel_high_index = -100
265.              }
266.          }
267.          //led.plot(count % 5, 1), //led.plot(count % 5, 2), //led.plot(count % 5, 3)
268.      }
269.
270.      //exercise---------------------------------------------------------------------------------------
271.      if (in_array_counter(last_analysed, C_MOVING,0) >= 300000 / interval) {
272.          analysed_array = {}
273.          analysed_array[time] = EXERCISE
274.          a_hi = false
```

```
275.          //led.plot(count % 5, 1), //led.plot(count % 5, 2), //led.plot(count % 5, 3)
276.
277.          //will ignore any anomalous pulse readings or spo2 readings once exercising - would be trigger
    ed by even walking up stairs
278.      }
279.
280.      //FALLS-------------------------------------------------------------------------------------
281.      if (analysed_array[time] == REST && a_hi == true && accel_high_index != -100) {
282.          if (accel_high_index < rest_index) {
283.              analysed_array[time] = FALL
284.              basic.clearScreen()
285.          }
286.
287.          if (accel_high_index == -100) { reset = true }
288.
289.      }
290.
291.      last_values = [array[time][x], array[time][y], array[time][z]]
292.      last_analysed[count % (accel_propty.movement_reset + 1)] = analysed_array[time]  //store the last
    5 analysed_accel_array values
293.      if (count % 5 == 0 && analysed_array[time] != FALL) { basic.clearScreen() } //when to clear screen
     for next LED sequence to occur
294.
295.      if (reset == true || a_hi == false) {
296.          resetter(false)
297.      }
298.
299.      return analysed_array
300.  }
301.
302.  function f_analyse(analysed_value: string, value: number, time: number, upper_norm: number, lower_norm
    : number) {
303.      analysed_value = 'norm'
304.      if (value > upper_norm) {
305.          analysed_value = 'hi'
306.          basic.showArrow(0)
307.      }
308.
309.      if (value < lower_norm) {
310.          analysed_value = 'lo'
311.          basic.showArrow(4)
312.      }
313.      return analysed_value
314.  }
315.
316.  //-------------------------------------------------------------------------------------------------------
317.  // MAIN flow
318.  loops.everyInterval(interval, function () {
319.      if (on == true) {
320.          pins.digitalWritePin(DigitalPin.P0, 0)
321.          //makes buzzer pulse if uncommented. If commented, astable circuit is required
322.          basic.pause(interval)
323.          time = Math.round(input.runningTime() / interval) * interval // rounds to nearest interval
324.          // {time: [X,Y,Z], time: [X,Y,Z] }
325.
326.          raw_accel_array[time] = [input.acceleration(Dimension.X), input.acceleration(Dimension.Y), inp
    ut.acceleration(Dimension.Z)]
327.          analysed_accel_array = f_analyse_accel(analysed_accel_array, raw_accel_array, time)
328.          if (analysed_accel_array[time] == FALL || button == true) {
329.              basic.showLeds(`
330.              #####
331.              #####
332.              #####
333.              #####
334.              #####
335.              `)
336.          }
337.
338.          raw_temp = input.temperature()
339.          analysed_temp = f_analyse(analysed_temp, raw_temp, time, temp_max, temp_min)
340.
341.          raw_pulse = f_get_pulse()
342.          if (rest == true) { //only measures when at rest - stationary arm
```

```
            raw_spo2 = f_get_spo2()
        }

        pulse_variability = f_pulse_variability(time, raw_pulse)
        analysed_pulse = f_analyse(analysed_pulse, raw_pulse, time, pulse_propty.upper_norm, pulse_pro
    pty.lower_norm)
        analysed_spo2 = f_analyse(analysed_spo2, raw_spo2, time, spo2_max, spo2_min)
        spo2_hypoxic_len = length_hypoxia(analysed_spo2, spo2_hypoxic_len)

        f_to_radio(time, analysed_accel_array, analysed_pulse + ';' + raw_pulse.toString() + ';' + pul
    se_variability, analysed_spo2 + ';' + raw_spo2.toString() + ';' + spo2_hypoxic_len.toString(), analysed_
    temp + ';' + raw_temp.toString(), button,)

        if (analysed_accel_array[time] == FALL) {
            pins.digitalWritePin(DigitalPin.P0, 1)
        } else {
            pins.digitalWritePin(DigitalPin.P0, 0)
        }

        analysed_accel_array = {}
        raw_accel_array = {}

        count++
    }
})
//-----------------------------------------------------------------------------------------------------

input.onButtonPressed(Button.A, function () {
    if (dementia == false) { button = true }
    //led.plot(0, 0), //led.plot(4, 4)
})

input.onButtonPressed(Button.B, function () {
    //led.plot(4, 0), //led.plot(0, 4)
    if (dementia == false) { resetter(true) }
})

interface IO_arrays { //input and output
    [key: string]: string[];
}

radio.onReceivedString(function (receivedString: string) {
    let edit_string = receivedString.split(',')
    if (parseInt(edit_string[0]) == control.deviceSerialNumber()) {
        let edit_array: IO_arrays = JSON.parse(edit_string[1])
        dementia = JSON.parse(edit_array['dementia'][0])
        spo2_min = parseInt(edit_array['spo2'][0])
        pulse_propty.lower_norm = parseInt(edit_array['pulse'][0])
        pulse_propty.upper_norm = parseInt(edit_array['pulse'][1])
        pulse_propty.rest = parseInt(edit_array['pulse'][2])
        on = JSON.parse(edit_array['on'][0])
    }
})

```

Appendix D: Receiver Code V3

[see description in code snippet]

```
//-------------------------------------------------------------------------------------------------
/*
 * Team: Fallarm
 * Project: Royal Society
 * Product: Fall prevention and detection (FPAD)
 * Version number: 3
 * File name: Fallarm-V3-Receiver
 * Description: Receives data from the sensor via radio and evaluates it, deciding if there is a fall
 *              or not, before passing to serial to be displayed on website using WebSerialAPI

List of shortened strings and their full counterparts:
    'int'        => 'Interpretation'
    'noc_hyp;'   => 'Nocturnal Hypoxia increases chance of fall;'
    'likely;'    => 'Fall likely;'
    'pulse_var;' => 'High pulse variation increases chance of fall;'
    'heatstroke;'=> 'Heatstroke likely'
    'med;'       => 'Medical action needed'
    'help;'      => 'Help requested
    'Norm'       => 'Normal'
    'L_pulse'    => 'Lower_pulse' (Lower bound)
    'U_pulse'    => 'Upper_pulse' (Upper bound)
    'U_sleep'    => 'Upper_sleep_pulse' (Upper bound of sleeping pulse)
Note that for the input, numbers 0 and 1 are used for booleans False and True
*/
//-------------------------------------------------------------------------------------------------

//CLASS SETUP------------------------------------------------------------------------------------
class People {
    name: string
    output: O_arrays
    input: I_arrays
    time: number
    constructor(name: string, output: O_arrays, input: I_arrays, time: number) {
        this.name = name
        this.output = output
        this.input = input
        this.time = time
    }
    serial_out() {
        return [this.time, this.name, this.output, this.input]
    }
}

interface People_interface {
    [key: number]: any //either People or 'null'
}

let People_array: People_interface = {}
let ids: number[] = []

function adder (id: number) {
    ids.push(id)

    for (let i = 0; i < ids.length; i++) {
        People_array[ids[i]] = new People('', {}, {
            'dementia': 0,
            'U_pulse': 60,
            'L_pulse': 100,
            'spo2': 90,
            'on': 1
            }, 0
        )
    }
}

adder(951260179)

```

```
68. //-----------------------------------------------------------------------------------------------
69.
70. function size(array: O_arrays) {
71.     let size = 0
72.     let my_keys = Object.keys(array)
73.     size = my_keys.length
74.     return size
75. }
76.
77. function nocturnal_hypoxia_id(spo2_radio: string[], sleep_count: number) {
78.     let noc_hypoxia = false
79.     if (sleep_count * 0.1 < parseInt(spo2_radio[2])) {
80.         noc_hypoxia = true
81.     }
82.     return noc_hypoxia
83. }
84.
85. function evaluate(radio_array: O_arrays, id: number) {
86.     let output_array: O_arrays = { 'state': [''], 'accel': [radio_array['accel'][0]], 'pulse': [radio_ar
    ray['pulse'][1]], 'spo2': [radio_array['spo2'][1]], 'temp': [radio_array['temp'][1]], 'int': ['']}
87.     // ^Copies the items relevant to the table headings^ to a separate array
88.
89.     //identifies if person is asleep:
90.     if ((current_hour > 10 || current_hour < 5) && (parseInt(radio_array['pulse'][1]) < People_array[id]
    .input['L_pulse'])) {
91.         sleep_counter ++
92.     }
93.
94.     //identifies if there are abnormal vitals (including nocturnal hypoxia)
95.     let vitals_array = ['temp', 'pulse', 'spo2']
96.     let nocturnal_hypoxia = nocturnal_hypoxia_id(radio_array['spo2'], sleep_counter)
97.     for (let i = 0; i < 3; i++) {
98.         if (i == 2 && nocturnal_hypoxia) {
99.             output_array['int'][0] += 'noc_hyp;'
100.              output_array['state'][0] = 'likely;'
101.          }
102.          else if (i == 1 && radio_array['pulse'][2] == 'hi') {// if pulse variability is too high
103.              output_array['int'][0] += 'pulse_var;'
104.              output_array['state'][0] = 'likely;'
105.          }
106.          else if (i == 0 && radio_array['temp'][0] == 'hi') {
107.              output_array['int'][0] += 'heatstroke;'
108.              output_array['state'][0] = 'likely;'
109.          }
110.          else if (radio_array[vitals_array[i]][0] == 'hi' || radio_array[vitals_array[i]][0] == 'lo') {
111.              output_array['int'][0] += (vitals_array[i] + ':' + radio_array[vitals_array[i]][1]) + ';'/
    /sets the state to the analysed pulse/spo2/temp array if they are abnormal
112.              if (output_array['state'][0] != 'likely;') {
113.                  output_array['state'][0] = 'med;'
114.              }
115.          }
116.      }
117.
118.      //identifies if a person is out of bed when should be sleeping
119.      if ((current_hour > 10 || current_hour < 5) && (radio_array['accel'][0] == 'continuous movement'))
     {
120.          output_array['state'][0] += 'Awake;'
121.      }
122.
123.      //identifies button press
124.      if (radio_array['button'][0] == 'true') {
125.          output_array['state'][0] += 'Help;'
126.          output_array['int'][0] += 'help;'
127.      }
128.
129.      //main identification stuff
130.      if (radio_array['accel'][0] == 'fall') {
131.          output_array['state'][0] = radio_array['accel'][0]
132.      }
133.      else {
134.          if (output_array['state'][0] == '') {
135.              output_array['state'][0] = 'Norm'
136.          }
```

```
    }

    return output_array
}

radio.onReceivedString(function (receivedString) {
    led.plot(2,4)

    let id = radio.receivedPacket(RadioPacketProperty.SerialNumber)
    id = 951260179
    People_array[id].time  = radio.receivedPacket(RadioPacketProperty.Time)

    let split_string: string[] = receivedString.split(',')

    radio_array[split_string[0]] = split_string[1].split(';')
        if (size(radio_array) == output_size) {
            People_array[id].output = evaluate(radio_array, id)
        }

    serial.writeLine(JSON.stringify(People_array[id].serial_out()))

    basic.clearScreen()
})

serial.onDataReceived(serial.delimiters(Delimiters.NewLine), function () {
    led.plot(2, 4)
    input_string = serial.readUntil(serial.delimiters(Delimiters.NewLine)).split(',')
    const action = 0, value = 1, id = 2
    interface p_i {
        [key: string]: number
    }
    const pulse_inputs: p_i = {'L_pulse': 0, 'U_pulse': 1, 'U_sleep': 2}

    if (input_string[action] == 'delete') {
        People_array[parseInt(input_string[id])] = 'null'
    }

    else if (input_string[action] == 'add') {
        adder(parseInt(input_string[id]))
    }

    else {
        People_array[parseInt(input_string[id])]['input'][input_string[action]] = input_string[value]
    }

    radio.sendString(input_string[id]+','+JSON.stringify(People_array[parseInt(input_string[id])]))
})

interface O_arrays { //output
    [key: string]: string[];
}
interface I_arrays { //input
    [key: string]: number
}

let input_string: string[] = []
let radio_array: O_arrays = {}
const output_size = 5 //ensures that everything has been sent before starting evaluation
radio.setGroup(101)

let current_hour = 0 //hour, e.g, 12 for 12:33
let sleep_counter = 0

```

Appendix E: (In progress) Website Code

This was a test to see how we could make a website to follow the required table.

```
<html>
<style>
table, th, td {
  border:1px solid black;
}
</style>

<script>
var person = ‘test’
</script>

<body>
<table>
   <tr>
     <th>Person</th>
     <th>State</th>
     <th>SpO2,</th>
     <th>Pulse</th>
     <th>Accelerometer</th>
     <th>Temperature</th>
     <th>Fallarm interpretation</th>
   </tr>
   <tr>
     <th id="a"></th>
     <script>
         document.getElementById('a').innerHTML = person;
     </script>
   </tr>

</table>
</body>
</html>

```

Appendix F: Roles

Person	Contribution to project
Alex Lodge	Built Astable circuit and documented it Evaluation Implementation into a care home setting Research into atrial fibrillation and dementia Documentation on problems (design board) Research into pulse oximeter (not documentation)
Hiranya Das	Display Board: -Design -Sensors -Specification PowerPoint: -Initial Ideas -Research -Design Development, Brief and Specification -Sensors Design: -Initial Case design Research: -Research into existing products Website: -Table of data -Auto-refreshing function (updates table) - Documentation
Umar Sultonov	Code for micro:bit sensor & receiver – V2 and V3 documented here Final case design Documentation for aims of project, its wider purpose and objectives Documentation of initial ideas and solutions (and evaluation of each) – considering sensors and their viability Table to show implementation of product in care home setting Research & documentation into body & ambient temperature's relevance; respiratory rate's relevance; blood pressure's relevance (dehydration, postural hypotension, and its 20% variation); etc. (all vitals research apart from atrial fibrillation and dementia) Research & documentation into accuracy of wearables measuring heart rate and blood oxygen Documentation of effect of nocturnal hypoxia (as part of documentation of blood oxygen level's relevance) Research into microcontrollers that could be used Research into available pulse oximeter sensors (Asking tech support for help with pulse oximeter)